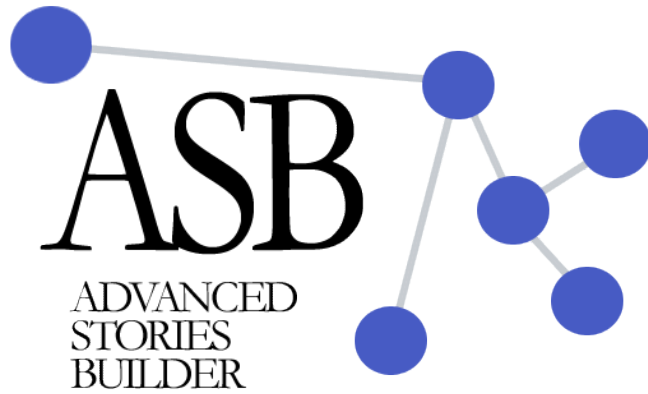




User Guide

Version 2.4.1

Copyright © 2007-2013 Katie Fletcher, Hartmut Koenitz, and Lee Adcock

Table of Contents

<i>Introduction.....</i>	<i>4</i>
<i>Some General Concepts.....</i>	<i>5</i>
<i>Quick Start</i>	<i>5</i>
<i>Feature Overview</i>	<i>6</i>
Splash Screen	6
Main Controls	7
Settings Screen	8
Props and Nodes Screen.....	9
Character Editor	11
Narrative Screen.....	12
Graph Story Screen.....	13
Graphics Editor Screen 1.....	14
Graphics Editor Screen 2: Text Placement.....	15
Graphics Editor Screen 3: Details	16
<i>Beat Functions</i>	<i>17</i>
General Beat Editing.....	17
TitleScreen	20
durScreen	21
IntroText	22
ConversationChoice	23
MovementChoice	24

PickProp	25
VideoBeat	26
SWFBeat.....	27
EndScreen	28
RandomTarget.....	29
SetGlobal	30
AddRemoveInventory	32
ConditionCheck	34
Sound in ASB	39
Clusters.....	40
<i>Tutorials</i>	<i>41</i>
Tutorial 1: Create a New Story	41
Tutorial 2: Edit Colors and Text Styles in a Saved Story.....	43
Tutorial 3: Adding Nodes and Props.....	46
Tutorial 4: Adding Characters.....	48
Tutorial 5: Editing Characters.....	50
Tutorial 6: Editing Visual Beat Content	54
Tutorial 7: Working in Graph View.....	59
<i>ASMLEngine Functions.....</i>	<i>61</i>
<i>FAQs and Best Practices</i>	<i>62</i>
<i>Definitions</i>	<i>65</i>
<i>Contact.....</i>	<i>69</i>

Introduction

The Advanced Stories Builder (ASB) allows you to quickly create interactive experiences such as games, interactive narratives, or training exercises. It can also be used as a rapid prototyping tool for projects that require more elaborate features. ASB requires no coding, and works with regular graphics, animation, and video file formats. Prototypes can be built entirely within ASB, without any additional files. The story files created in ASB are written in an XML-based markup language (ASML), which can then be played via an ASMLEngine that operates in Flash.

The Advanced Stories Authoring And Presentation System, or ASAPS, originated from the Advanced Stories Group (ASG) at Georgia Tech, which was founded by Hartmut Koenitz in 2005 and supervised by Dr. Janet Murray. The goal of the ASG was to extend the practice and understanding of interactive storytelling by creating a formal structure for describing and authoring interactive stories, and to create exemplary stories that illustrate the expressive possibilities of the form. In creating ASAPS, Dr. Koenitz asserts that interactive stories are a new format of expression that shares some of the characteristics of existing story formats and some of the characteristics of computer games, but has its own affordances and its own potential for capturing human experience.

To create a story, authors combine "beats", or atomic narrative units, to create a complete interactive experience. The 14 different beat types include static elements like a title screen, as well as flexible elements, which contain choices in a conversation, for navigation, or for adding items to an inventory. Finally, procedural beats allow authors to manipulate counters, global variables, inventory items, and timers. This inventory of elements, together with audiovisual assets, allow for a wide range of visual styles, such as film-noir, horror, superhero comic, and fantasy.

ASAPS is an ongoing research project. An iOS playback engine is currently in late stage development.

Some General Concepts

There are a few general concepts in the *AdvancedStories* system. Understanding these concepts makes it easier to work with ASB:

- **Document** and **Engine**: stories created in ASB are saved as ASML (Advanced Stories Markup Language) in a file called *Story.xml*. Additional files (graphics, videos, animations) are linked to from the story document. The engine reads the story file, finds the referenced graphics/videos/animations, and creates the experience for the interactor. This is similar to how web browser work.
- **Props**: Like in drama, props can be everything from a knife to a spaceship. The interactor can add props to her/his inventory
- **Nodes**: backdrops for visual beats, can be shared by many different beats.
- **Characters**: the characters in your story.
- **Beat**: The smallest functional unit in a story generated with ASB. In other words: you create, edit, and link beats to produce stories

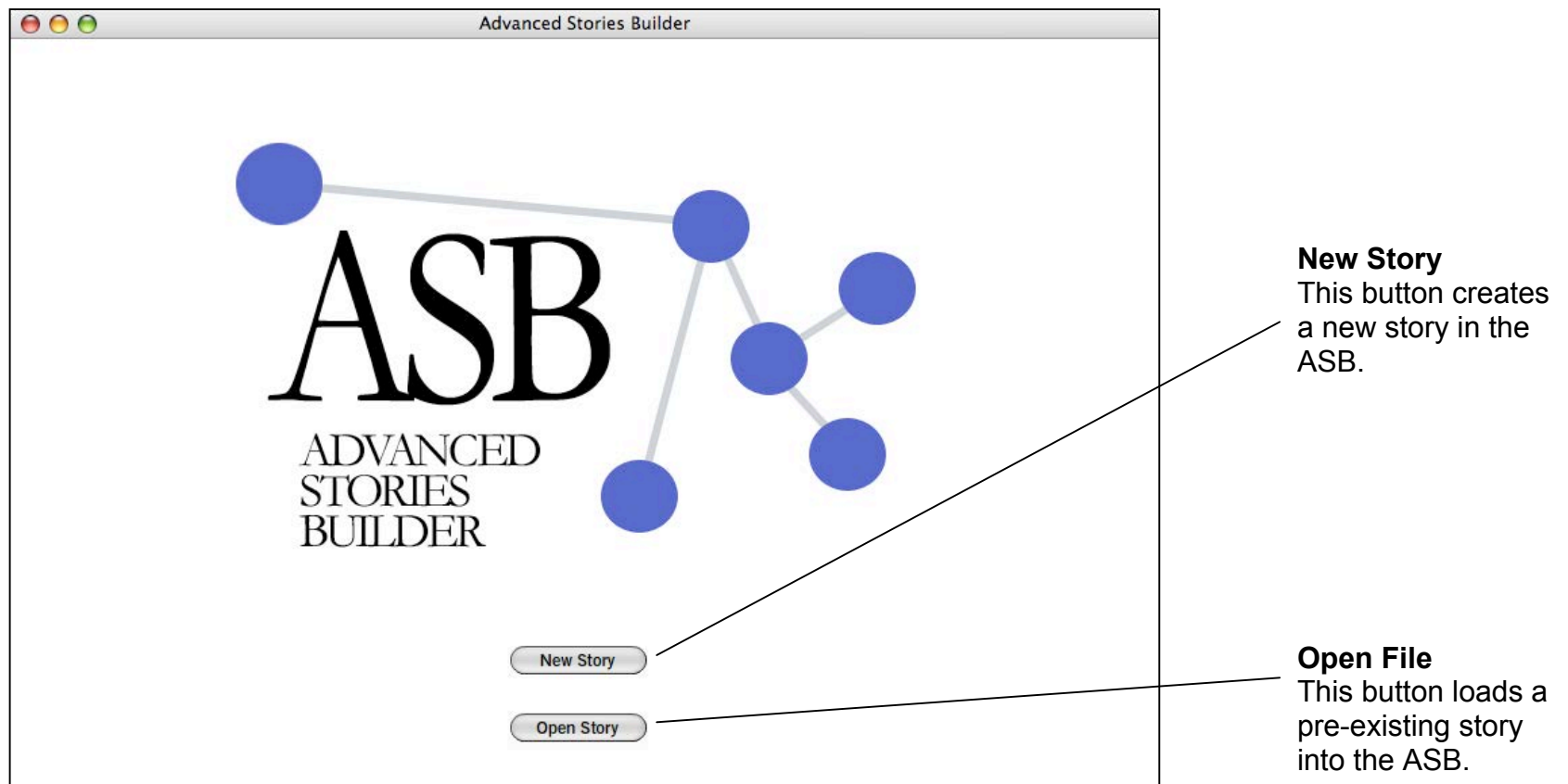
To learn more about the terms and definitions used in ASB, have a look at chapter 5: *AdvancedStories Definitions*.

Quick Start

If you want to dive right in, skip the Overview section and go directly to the tutorials. You may want to come back to the overview later, to learn more about additional features available in ASB.

Feature Overview

Splash Screen



Main Controls

Story/Creator Name

Type the name of your story in the first field and the creator in the second one. Both will appear automatically on the title screen of your story.

Settings Tab

Click this tab to go to the Settings screen, with debugging and text box tools.

Nodes/Props Tab

Click this tab to go to the Nodes and Props screen, where you can add and edit the objects in your story and the node backgrounds.

Characters Tab

Click this tab to go to the Character screen, where you can add and edit characters and counters.



Narrative Design Tab

Click this tab to go to the Narrative Design screen, where you can add and edit story beats and the graphical elements of the story.

Test Story

This launches the current story in the engine.

Undo/Revert

Undo removes the last change you made, revert goes back to the last saved version.

New Story

Use this to start a new project.

Open Story

Use this to open an existing project.

Save Story

This button saves the current story.

Save Backup Copy

Saves the current story file under a new name in the folder you select.

Settings Screen

Debugger

You can start the story at a specific beat by changing the value in the “firstbeat” field.

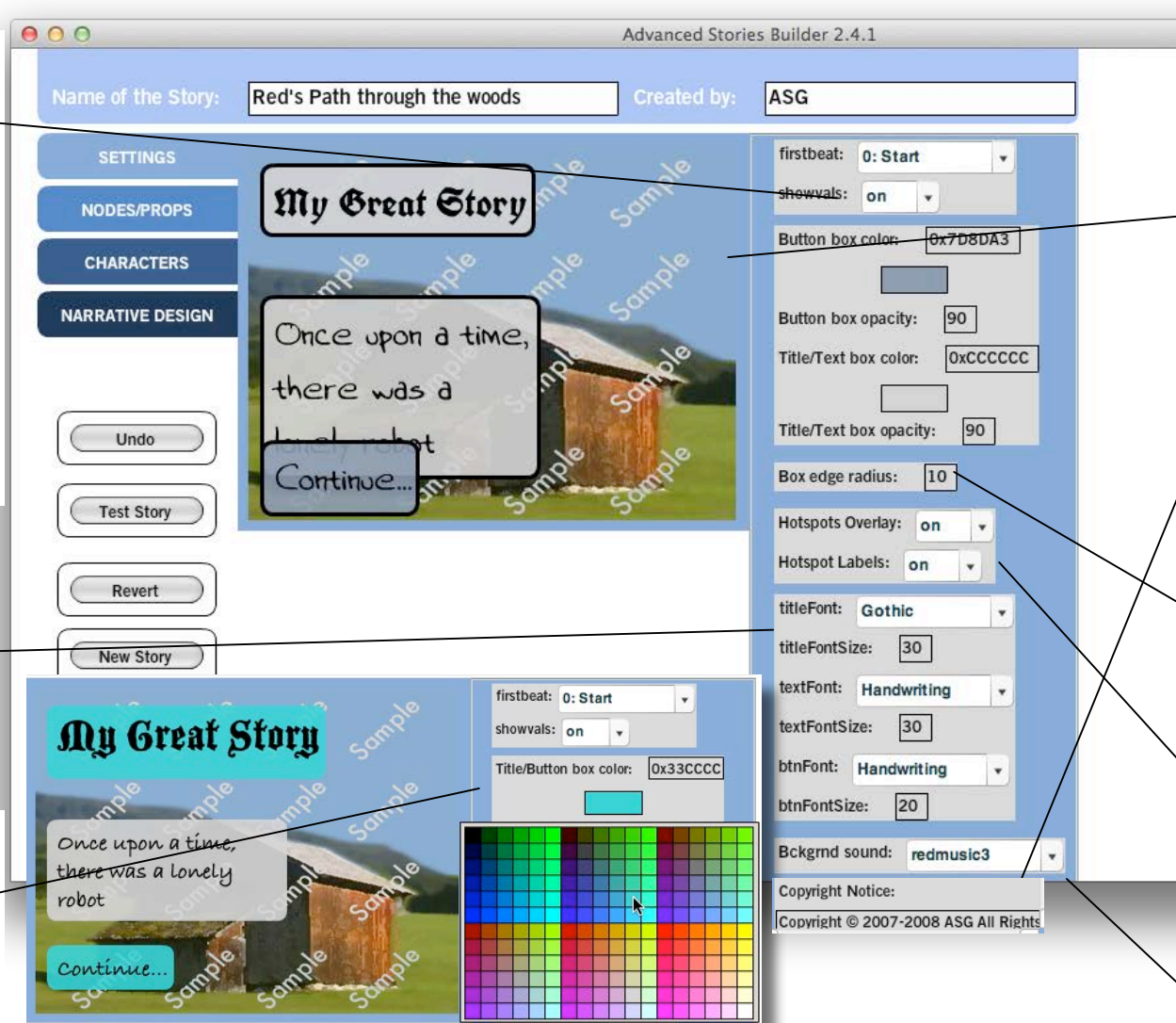
The default setting displays the debugging information in the corner when the story runs. You can hide this by changing the “showvals” field value to “off.”

Fonts

You can select different engine fonts for the text in each of the 3 types of boxes.

Background Color, Story Text Boxes

You can change the opacity and color of text boxes here. Click on the color swashes to select a color (see inset). You can also input a hexadecimal color



Text Preview

Preview your font and color choices as they will display in the story.

Copyright Notice

This copyright notice will be saved in your story file. The normal form is “Copyright © Year Author All Rights reserved”
The copyright notice is not visible in the engine.

Box Shape

You can change the amount of curve in the corner radius of the text boxes.

Hotspots

You can decide if hotspots are visible and if the labels are displayed.

Bckgrmd Sound

This allows you to set background music for the entire story.

Props and Nodes Screen

Prop

Each prop added to the story appears in a light grey box like this.

- prop id: the identification number unique to each prop; automatically added by system
- prop name: the name you give it
- Asset File: the name of the graphics file

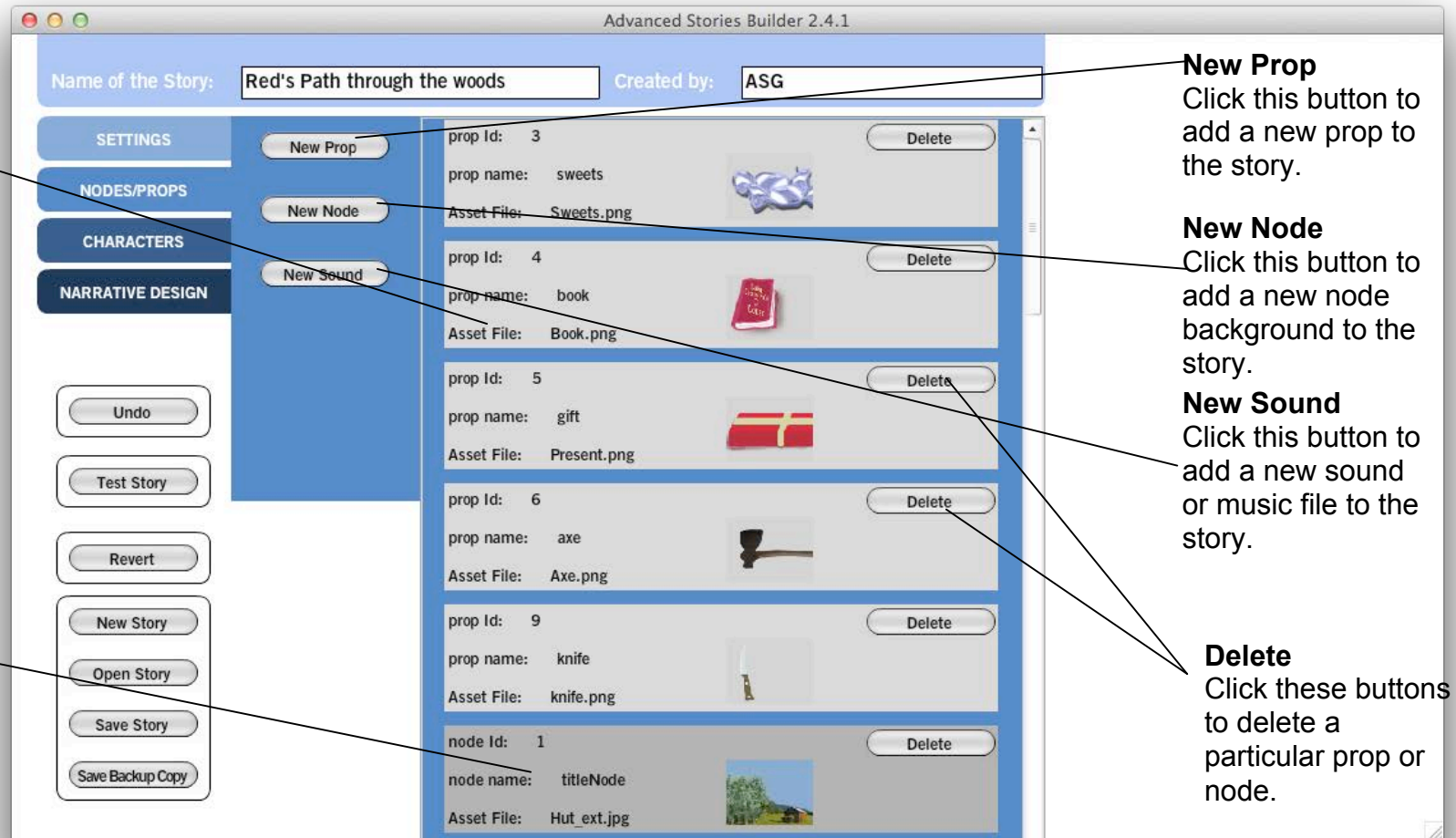
Node

Each node added to the story appears in a dark grey box like this.

node id: the identification number unique to each node; automatically added by system

node name: the name you give the node

Asset File: the name of the graphics file



Characters Screen

Character

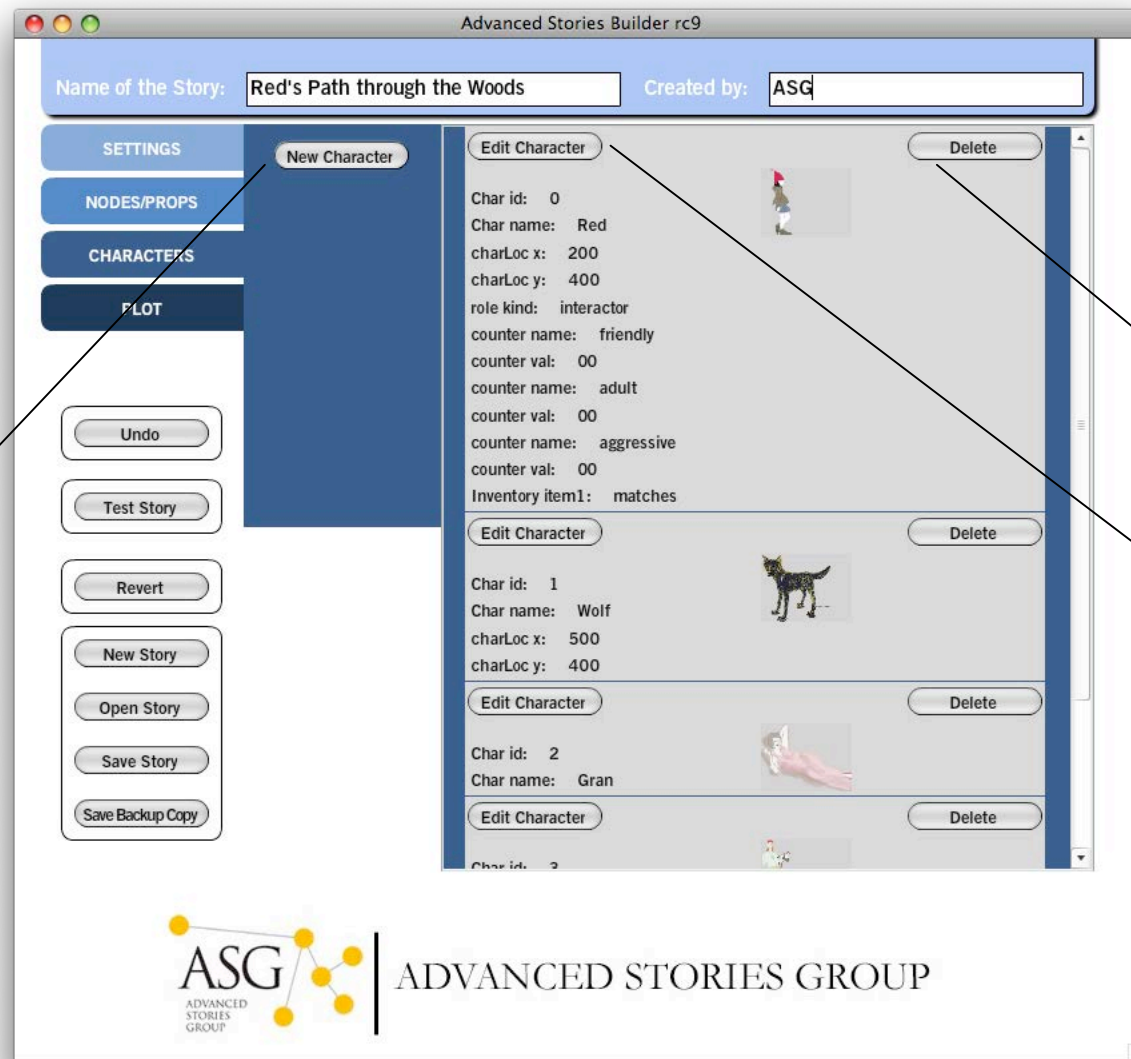
Each character added to the story appears in a box like this.

Char id: the identification number unique to each character; automatically added by system

Char name: the name you give the character

New Character

Click this button to go to add a new character with the Character Editor.



Delete

Click these buttons to delete a particular character.

Edit Character

Click this button to go to the Character Editor, which enables you to add graphics, states, counters, and items to that character's inventory.

Character Editor

Character Name/ID

You can change the character's name here.

The ID is assigned by the system and should not normally be changed.

Inventory

Click this button to add an Item to the character's inventory

Save/Cancel;

Use these buttons to leave the character editor

Counters

Click this button to add a counter to the character. You need to specify a name and an initial value.

Advanced Stories Builder rc6

Character Editor

Character ID: 0

Character Name: Red

This is the Interactor's character

x: 200

y: 400

Inventory of this Character:

item1:	matches
Delete Item	

Add Item

States of this Character:

State Name:	default	Delete State
Asset File:	Redsm.png	

State Name:	right	Delete State
Asset File:	RedSmR.png	

State Name:	attacking	Delete State
Asset File:	LLRH_fightSM.png	

State Name:	attackingAxe	Delete State
Asset File:		

Add State

Counters of this Character:

name:	friendly	val:	00	Delete Counter
name:	adult	val:	00	Delete Counter
name:	aggressive	val:	00	Delete Counter

Add Counter

Save Character

Cancel

Interactor role

One character must be designated the interactor's character. Once it is set, the button disappears for this character

States

You add and delete states (different graphical representations of a character) here

Narrative Screen

Edit

Click this button to edit the beat, including the type of beat, adding text, and which beat should follow this one.

Copy

Click this button to copy this beat. A new beat with the same parameters and content will be added to the list

Delete

Click this button to delete this beat from the story.

Cluster List

This displays the clusters that you've created as folders. Click each folder to see the beats you've placed in each cluster.

Beat

Each beat added to the story appears in a box like this.

Beat id: the identification number unique to each beat; automatically added by system

Beat name: the name you give the beat

New Beat

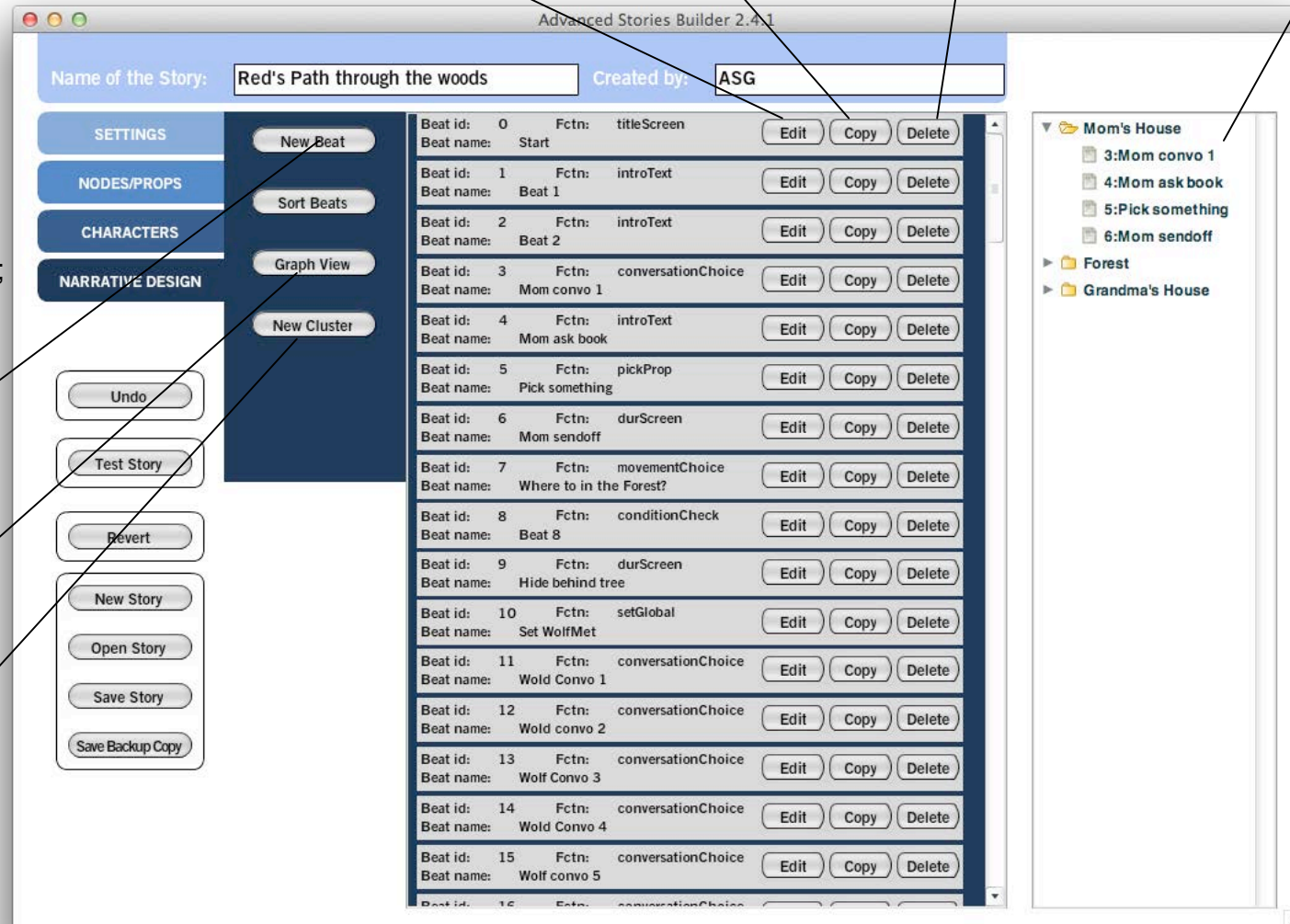
Click this button to add a new beat to the story.

Graph Story

Click this button to generate a graph of the story.

New Cluster

Click this button to create a cluster (see next page).



Graph Story Screen

Beat Map

The ASB maps out the beats currently in your story. Different colors represent different beat functions like ConversationChoice or IntroText.

You can edit Beats by clicking the beat symbols

New Beat

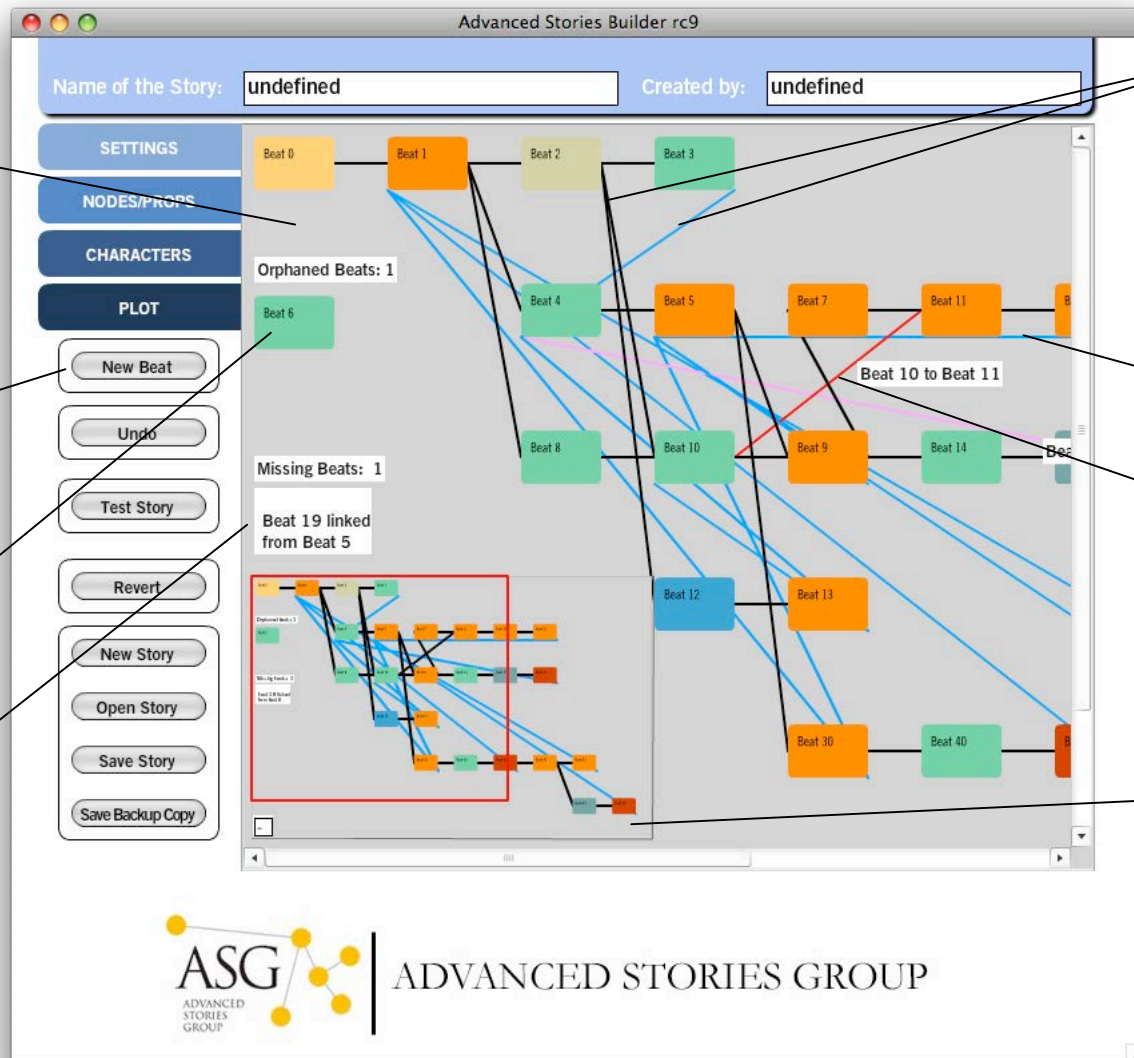
Click this button to add a new beat to the story in the Story Graph view.

Orphaned Beats

This notifies you of any orphaned beats. These are beats that are not connected to any others.

Missing Beats

This notifies you of any missing beats. These happen when you specify a target beat, when that beat does not yet exist.



Beat Connections

These lines indicate connected beats. The incoming connection attaches at the left-hand, while the line coming out of the right-hand side leads to the target beat.

Blue connections mark returns

Mousing over connections result in a pop-up showing the names of the beats connected. Also the connection is highlighted

Navigation Map

This shows a miniature overview of the story graph. The red rectangle depicts your current location. Click the +/- in the lower left corner to hide/show the map

Graphics Editor Screen 1

Graphics Editor Screen

The Graphics Editor lets you arrange the visual elements of a beat. Move Props/Characters and Hotspots around to change their location

Interactive Hot Spots

You can place interactive fields over the background for PickProp and MovementChoice beats. New hotspots are created with the button below.

Placing Props/Characters

You can view the available props/characters here. Click on the thumbnail image to place it on the beat screen.

Available Nodes

You can view the available nodes here. Click on the thumbnail image to place it on the beat screen.

Edit Beat 7 Where to in the Forest?



Prop Control Options

You can control the size of a prop by typing in the size field. You can remove the prop by clicking the delete button. You can also determine its "depth" in the scene with the z field.

Hotspot Control Options

Hotspots can also be named for easier referencing

Character Control Options

For a character, you can choose the state in a pop-up in addition to size and location

Save

Click the Save button when you are done editing your beat visuals.

Cancel

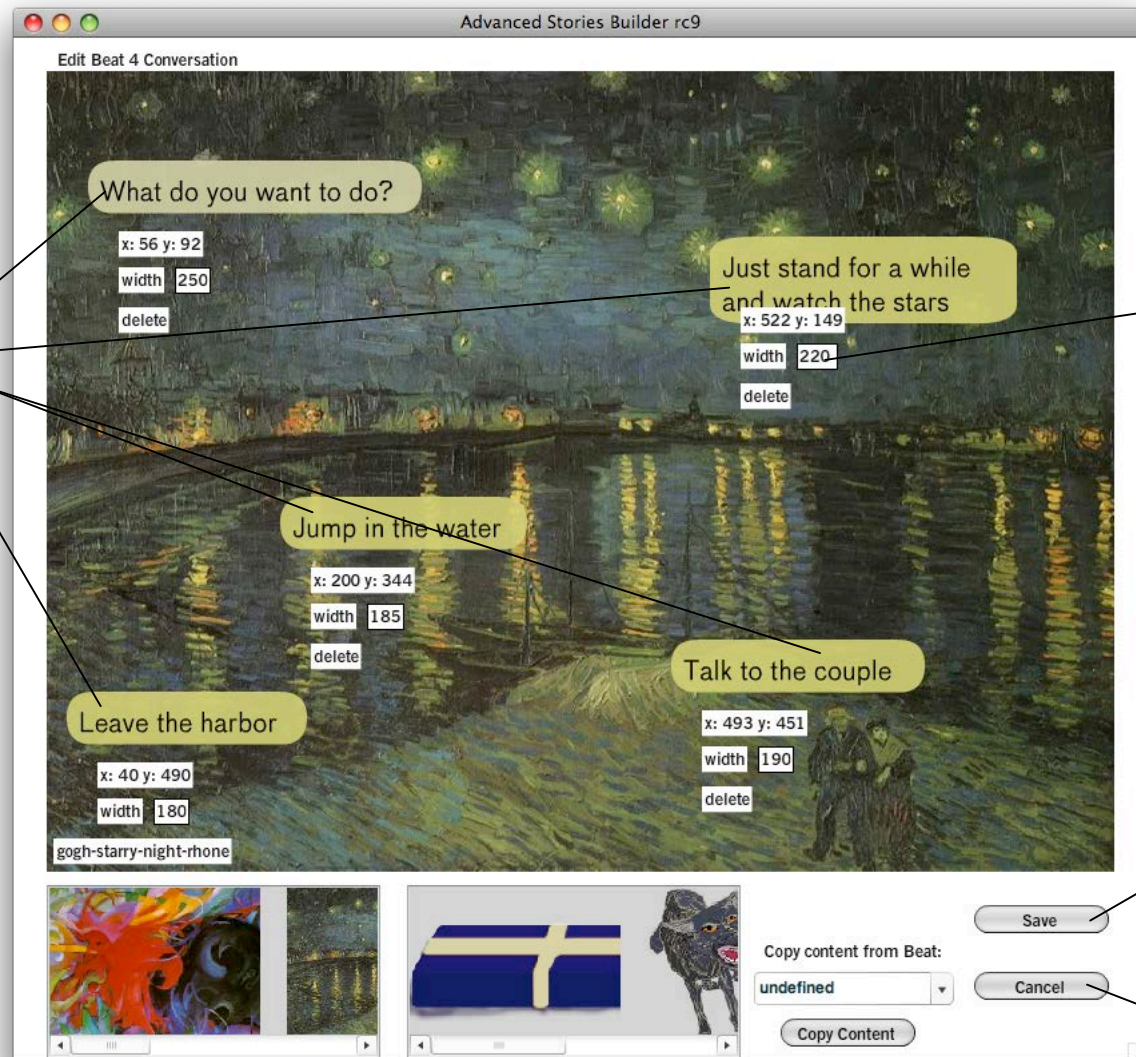
Click the Cancel button to go back to the beat without saving.

Graphics Editor Screen 2: Text Placement

Graphics Editor Screen Text Placement

The Graphics Editor lets you arrange text boxes on a node by dragging or changing the X and Y values. Also, you can change the width of textboxes. Text flow will automatically adjust to the new width.

Hint: You only adjust the look and placement of textboxes here. Text itself is entered the respective beat's Edit window (see next section)



TextBox Width
Change this number to adjust the width of the textbox.

Save
Click the Save button when you are done editing your beat visuals.

Cancel
Click the Cancel button to go back to the beat without saving.

Graphics Editor Screen 3: Details

Graphics Editor Screen Details

The Graphics Editor displays useful information and lets you copy content from other beats

Current Beat ID und Name

The ID and name of the current Beat are displayed in the upper left corner.

Current Node

The name of the currently placed node is displayed in the lower left corner when you mouse over a node.

Node Name

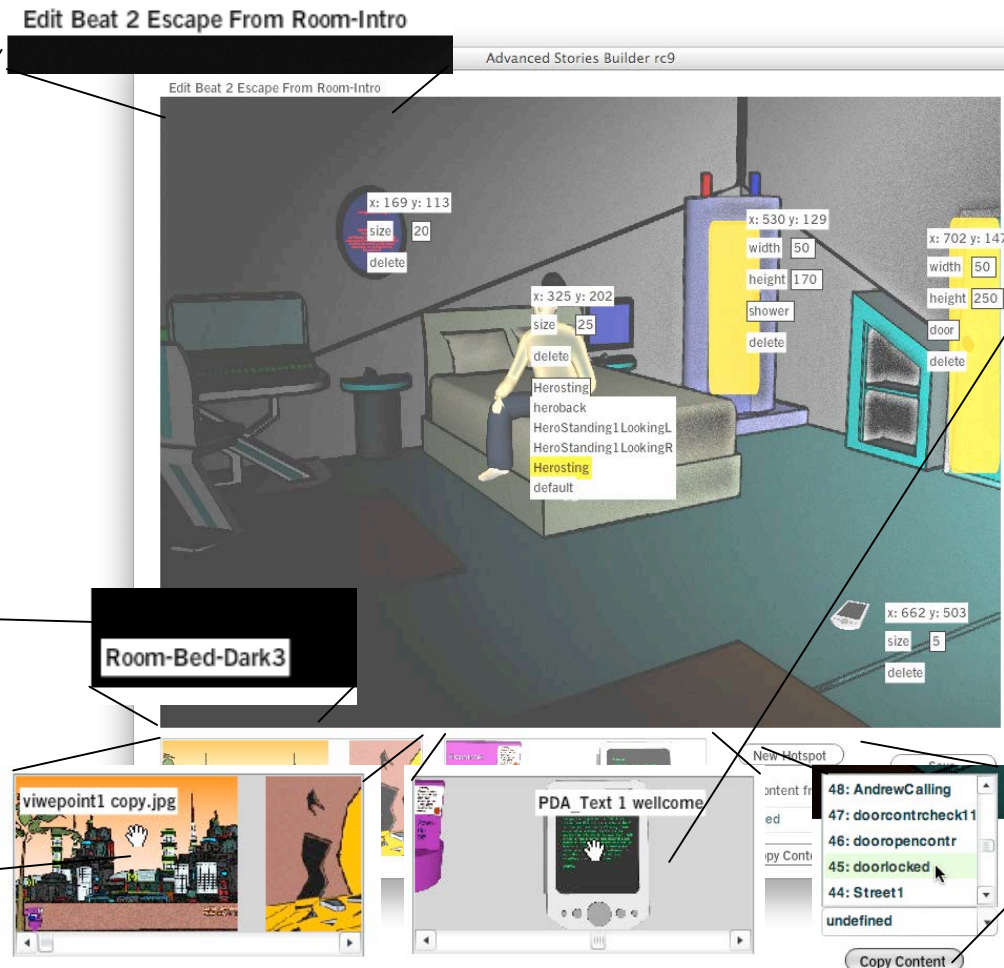
Mousing over a node shows the name of the node.

Prop/Character Name

The name of a prop or character is displayed when you mouse over.

Copy Content

To copy content from another beat, select a beat in the pop up and click the button *Copy Content*



Beat Functions

AdvancedStory projects divide narrative and functional action into discrete units called “beats.” When used as narrative structure, a beat contains one moment in time (such as a decision). A beat also performs one specific action, which may be hidden from the interactor. There are 12 different kinds of beats, or beat *functions*: TitleScreen, IntroText, durScreen, ConversationChoice, MovementChoice, PickProp, SetGlobal, ConditionCheck, VideoBeat, SWFBeat, randomTarget and EndScreen.

Each beat type has different variables and options that affect how it operates and appears in the story. There are two main types of Beats – visible ones and invisible ones. Visible beats are TitleScreen, IntroText, durScreen, ConversationChoice, MovementChoice, PickProp, VideoBeat, SWFBeat, and EndScreen. Invisible Beats are SetGlobal, randomTarget and ConditionCheck. The visible Beats present content to the user, while invisible beats perform behind the scene setting of variables and conditional branching.

General Beat Editing

New beats are created by clicking the *New Beat* button in the Plot screen. We recommend you change the default Beat Name for each beat. The options in the Beat window change depending on the type of the beat. Click on *Save New Beat* to add the beat to the list of beats on the right. You can edit beats by clicking the *Edit* button in the respective beat’s box. This will open the *Edit Beat* window. The appearance of the *Edit Beat* window is different for each Beat type, the different windows are shown on the following pages.

Adding Visual Content

While you can build a text-based in ASB story without adding any graphics, visual content greatly enhances the expressive options available to you as the creator.

Size and File Formats

Before you import any graphics into ASB, take note of the following specifications for graphics, videos and animations. For props and characters, both file formats support transparency, so that your objects blend seamlessly with the background. Most modern image editor programs allow you to create PNGs with transparencies, for example Adobe Photoshop.

Nodes (Backgrounds):

Size: 800x600 pixels

File formats: JPEG, SWF (Flash animation), FLV (Flash video file)

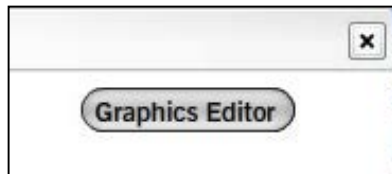
Props/Characters:

Size: can be scaled in ASB, but should normally be smaller than 800x600 pixels

File formats: PNG with transparency, SWF (Flash animation)

Editing visual beat content

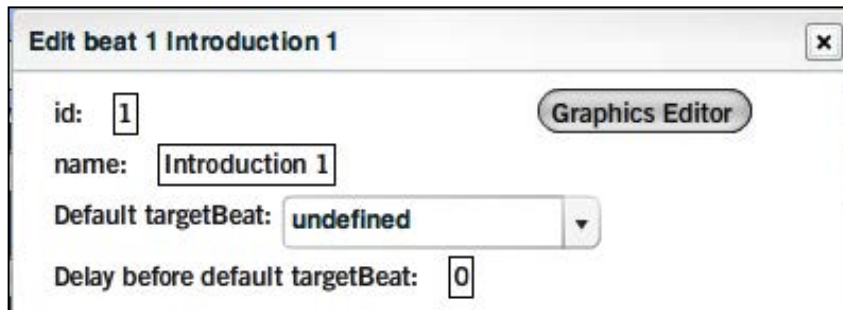
Before you can add graphics to a beat, you need to bring it into ASB in the form of Props, Nodes, and Characters. Please refer to the tutorials on how to accomplish this. Once you have added graphics, you can click the *Graphics Editor* Button in any visual beat to add visual content to a beat – see tutorial 6.



Default targetBeats

Several beat types (*TitleScreen*, *introText*, *ConversationChoice*, *MovementChoice*, *PickProp*, *EndScreen*) can have a *default targetBeat*. A default target beat becomes active after a set time delay, which means the story will jump to this beat automatically. This creates interesting possibilities for the flow of your story. Essentially you can put the interactor under time pressure to make a decision. You can also make a story that runs on its own, as long as the interactor does not interrupt.

Note: The *default targetBeat* functionality is optional, if you don't want to use it, leave the standard values (undefined and 0) alone. You can define for each beat individually if you want a default beat or not.



The screenshot shows a window titled "Edit beat 1 Introduction 1" with a close button (X) in the top right corner. Inside the window, there are four fields: "id:" with a text box containing "1", "name:" with a text box containing "Introduction 1", "Default targetBeat:" with a pull-down menu showing "undefined", and "Delay before default targetBeat:" with a text box containing "0". A "Graphics Editor" button is located to the right of the "id:" field.

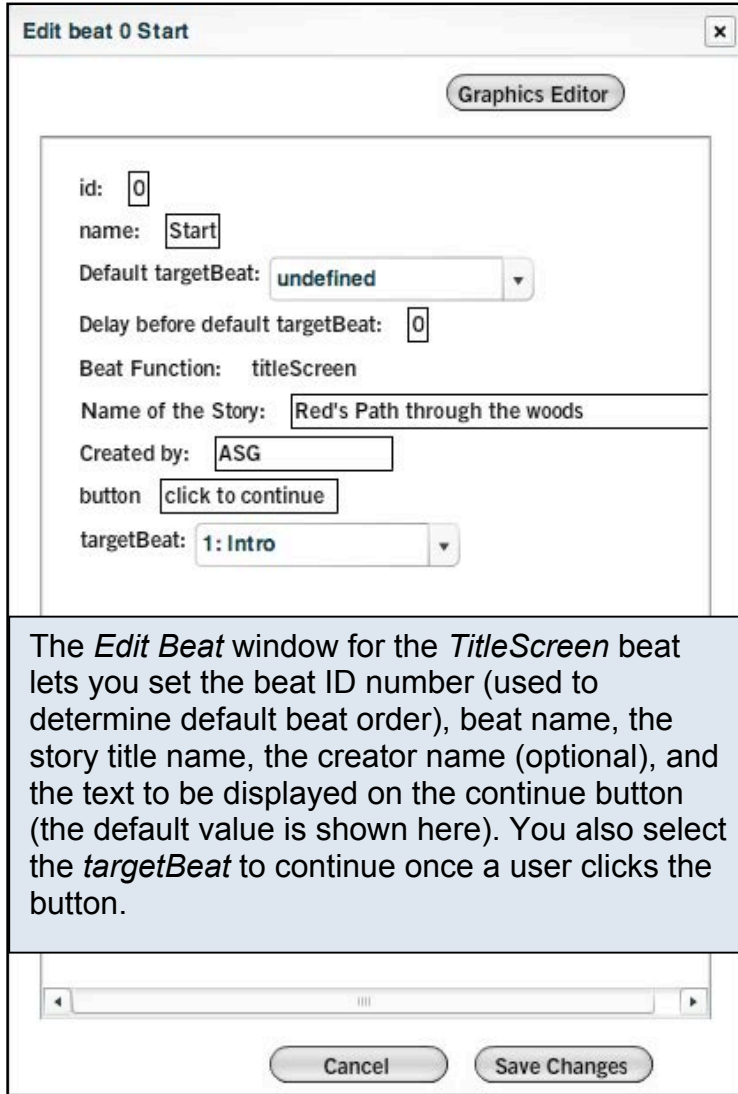
The upper part of an *Edit Beat* window with the *default targetBeat* section.

Select the destination for the default targetBeat from the pull-down menu.

Change the delay to something other than 0. This value is in milliseconds, so 1000 equals 1 second.

TitleScreen

The *TitleScreen* Beat automatically appears when you start a new story. It may be deleted if you do not want to start your story with a title screen. You can use the Graphics Editor to add images or video that you have uploaded as nodes or props. Title and creator font appearance is edited in the Settings screen.



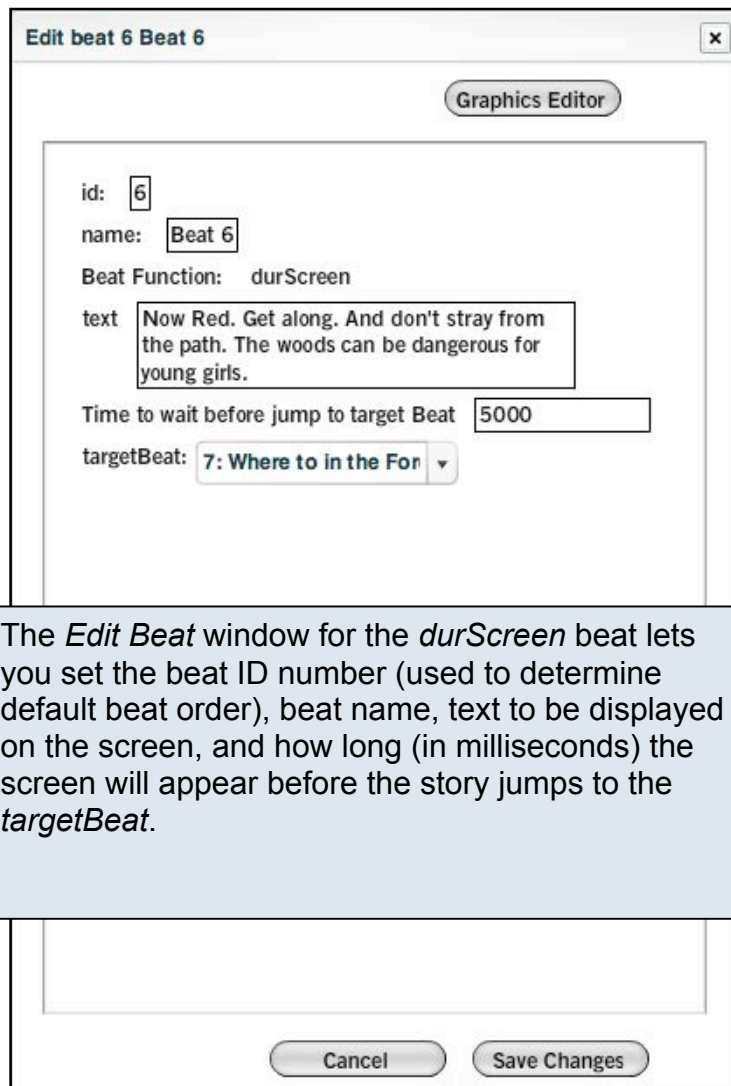
The *Edit Beat* window for the *TitleScreen* beat lets you set the beat ID number (used to determine default beat order), beat name, the story title name, the creator name (optional), and the text to be displayed on the continue button (the default value is shown here). You also select the *targetBeat* to continue once a user clicks the button.



This Title Screen displays the values entered in the *TitleScreen* beat window at left.

durScreen

The *durScreen* beat function displays a screen for a set amount of time, after which the engine automatically moves to the target beat. You can place a node and props or characters in a *durScreen* beat using the Graphics Editor. The *durScreen* is useful to control the pace of the story, and to dislocate or interrupt the interactor's feeling of control over it.



Edit beat 6 Beat 6

Graphics Editor

id:

name:

Beat Function: durScreen

text

Time to wait before jump to target Beat

targetBeat:

Cancel Save Changes

The *Edit Beat* window for the *durScreen* beat lets you set the beat ID number (used to determine default beat order), beat name, text to be displayed on the screen, and how long (in milliseconds) the screen will appear before the story jumps to the *targetBeat*.



IntroText

The *introText* beat function displays a screen with text. The story pauses until the interactor clicks the continue button, after which the story jumps to the next beat. You can place a node background and props or characters in an *introText* beat using the Graphics Editor. The *introText* function is useful for large sections of narrative text, and for maintaining the interactor's interactive relationship with the story.

Edit beat 1 Introduction 1

id: Graphics Editor

name:

Default targetBeat:

Delay before default targetBeat:

Beat Function:

intro

button

targetBeat:

Cancel Save Changes



The *Edit Beat* window for the *introtext* beat lets you set the beat ID number (used to determine default beat order), beat name, a *Default targetBeat*, and the *Delay* before the default targetBeat will become active. The *intro* field contains the text displayed on the screen, and the text that appears in the continue button that jumps to the next beat. Additionally you need to select the targetBeat from the pull-down menu.

ConversationChoice

The *conversationChoice* beat function displays a screen with a question and up to three possible responses from which the interactor may choose. You can place a node background and props or characters in an *conversationChoice* beat using the Graphics Editor. This beat function implements powerful interactive elements that can affect the narrative in ways not obvious to the interactor. The response choices can lead to different beats, but each response can also change counters that determine conditions and thresholds connected to additional opportunities later in the narrative.

Edit beat 15 Beat 15

Graphics Editor

name: Conversation with wolf 3

Default targetBeat: undefined

Delay before default targetBeat: 0

Beat Function: conversationChoice

questioner: 1

question: Now you're making me blush.

Delay before choices appear: 1500

Choice ID 1:

content: Perhaps I could offer you a treat?

Counter to change: adult Value: 03

targetBeat: 20: Beat 20

Add Choice

Choice ID 2:

content: I have a question for you.

Counter to change: aggressive Value: 02

targetBeat: 25: Beat 25

Remove Choice

Add Choice

Cancel

Save Changes

The *Edit Beat* window for the *conversationChoice* beat lets you set the beat ID number (used to determine default beat order), beat name, *defaulttarget* options, the ID of the character posing the question, the question text, the delay between when the question appears on the screen and when the answers appear (in milliseconds), and conversation choice(s). You can add more choices or remove existing ones with the *Add Choice* and *Remove Choice* buttons. You can edit the content text for each response choice, as well as the counters and values they may affect and the different target beats to which they jump if selected by the interactor.



MovementChoice

The *MovementChoice* beat function displays a screen with up to three places (hotspots), props or characters, which the interactor may move to by clicking. A *MovementChoice* beat requires you to first use the Graphics Editor to place characters, props, or hotspots on the Node. To return to this window, click *Save* in the Node editor. Now your locations are available in the locations pulldowns.

Similar to *ConversationChoice*, a click on the different locations in *MovementChoice* can lead to different beats, but each response can also change counters that determine conditions and thresholds connected to additional opportunities later in the narrative.

Edit beat 7 Where to in the Forest?

Graphics Editor

name: Where to in the Forest?

Default targetBeat: undefined

Delay before default targetBeat: 0

Beat Function: movementChoice

questioner: 1

question: Where should I go now?

Choice ID 1:

Location: tree

Counter to change: aggressive Value: 01

targetBeat: 8: Beat 8

Add Choice

Choice ID 2:

Location: wolf

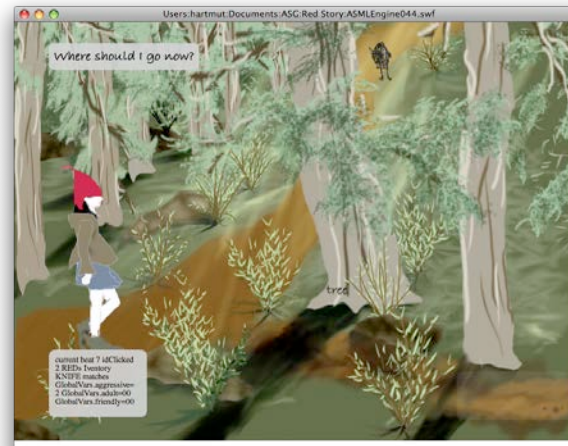
Counter to change: adult Value: 02

targetBeat: 8: Beat 8

Remove Choice Add Choice

Cancel Save Changes

The *Edit Beat* window for the *MovementChoice* beat lets you set the beat ID number (used to determine default beat order), beat name, *defaulttarget* options, the character posing the question (leave at "1"), the question text, and the location choices (created in the Graphics Editor). You can also set counter changes and different target beats for each choice. You add more choices or remove existing ones with the *Add Choice* and *Remove Choice* buttons. You create the location choices in the Graphics Editor. The name of each choice will appear in the *Location* dropdown menus.



PickProp

The *PickProp* beat function displays a screen with up to three props or characters, which the interactor may pick by clicking. A *PickProp* beat requires you to first use the Graphics Editor to place the props on the Node. To return to this window, click *Save* in the Node editor. Now your props are available in the locations pulldowns.

Similar to *ConversationChoice* and *MovementChoice*, a click on the different props in *PickProp* can lead to different beats, but each response can also change counters that determine conditions and thresholds connected to additional opportunities later in the narrative.

id: 5

name: Beat 5

Default targetBeat: undefined

Delay before default targetBeat: 0

Beat Function: pickProp

questioner: 1

question: And before kissing her mother goodbye, Red adds to her basket:

Choice ID 1:

Location: undefined

desc: Some pinata candy

Counter to change: friendly Value: 02

targetBeat: 6: Beat 6

Add Choice

Choice ID 2:

Location: knife

desc: A knife

Cancel Save Changes

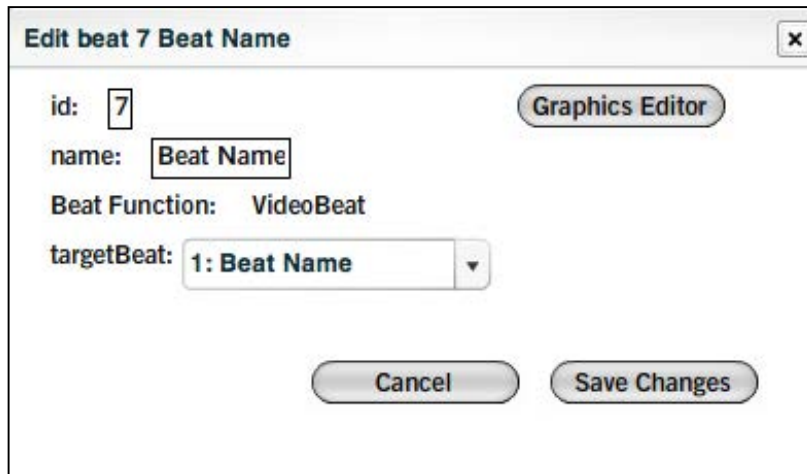
The *Edit Beat* window for the *PickProp* beat lets you set the beat ID number (used to determine default beat order), beat name, *defaulttarget* options, the character posing the question (leave at "1"), the *question*/description text, and the choices of props to pick (created in the Graphics Editor). The *desc* for each prop appears if the interactor mouses over that prop. You can also set counter changes and different target beats for each choice. You can add more choices or remove existing ones with the *Add Choice* and *Remove Choice* buttons. You create the location choices in the Graphics Editor. The name of each choice will appear in the *Location* dropdown menus.



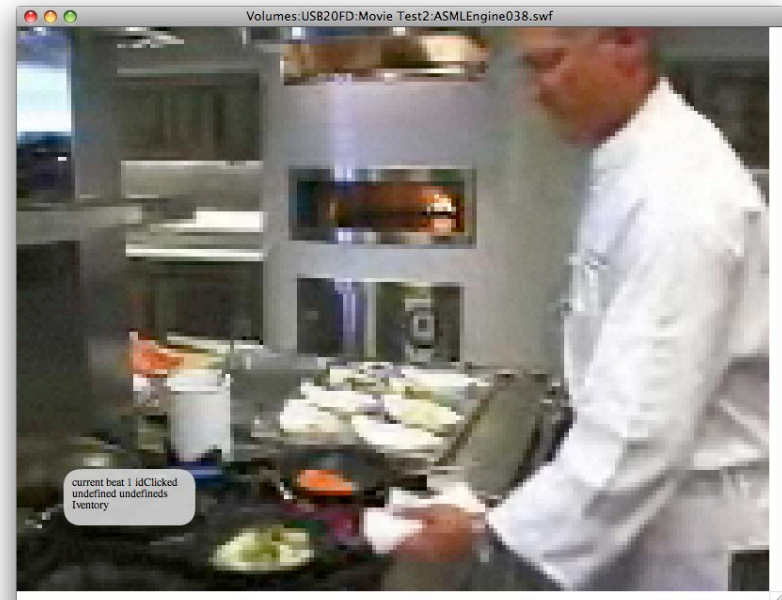
VideoBeat

The *VideoBeat* beat function displays a screen running a video from start to end. Once the video has finished playing, the narration continues to the defined targetBeat.

Tip: Use this beat only if you only want to have a video running and no interactivity. Otherwise you should use any of the other visible beats with the video as the node (background).



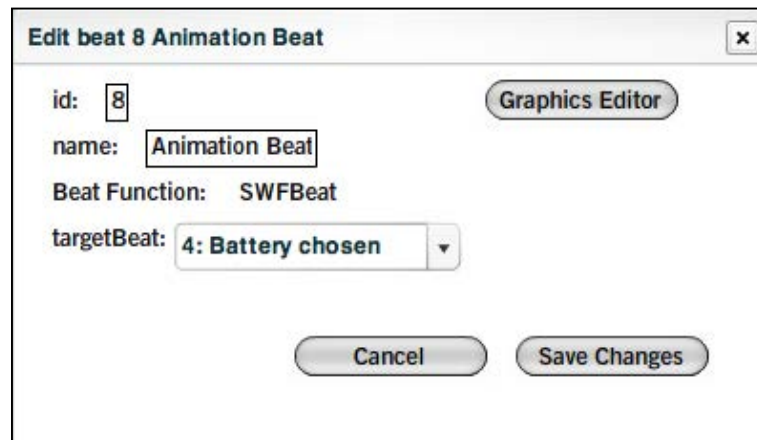
The *Edit Beat* window for the *VideoBeat* beat lets you set the beat ID number (used to determine default beat order), and beat name. The targetBeat will be branched to as soon as the video has finished playing.



SWFBeat

The *VideoBeat* beat function displays a screen running a Flash animation from start to end. Once the animation has finished playing, the narration continues to the defined *targetBeat*.

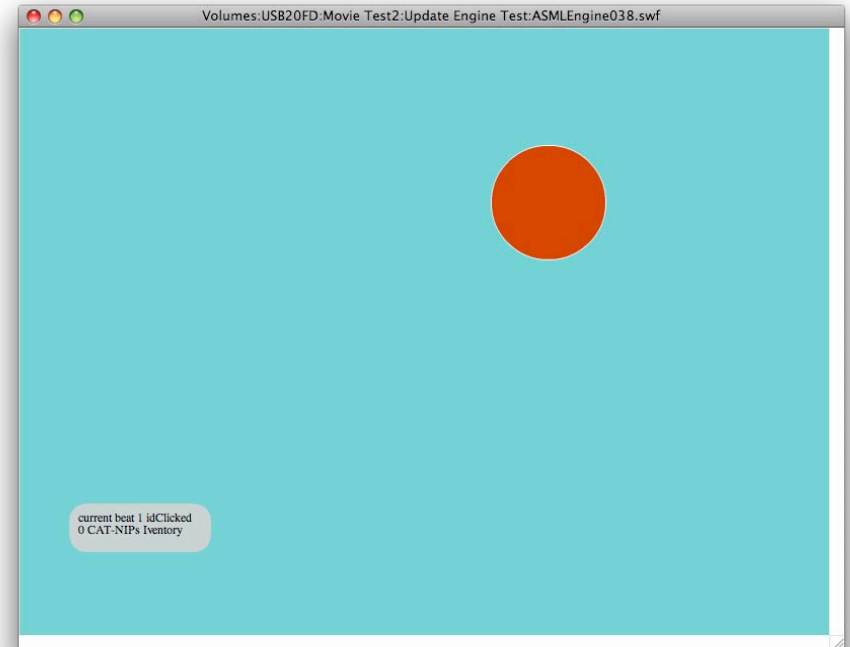
Tip: Use this beat only if you only want to have an animation running and no interactivity. Otherwise you should use any of the other visible beats with the animation as the node (background).



The dialog box titled "Edit beat 8 Animation Beat" contains the following fields and buttons:

- id:** A text box containing the number "8".
- name:** A text box containing "Animation Beat".
- Beat Function:** A dropdown menu showing "SWFBeat".
- targetBeat:** A dropdown menu showing "4: Battery chosen".
- Buttons:** "Graphics Editor", "Cancel", and "Save Changes".

The *Edit Beat* window for the *SWFBeat* beat lets you set the beat ID number (used to determine default beat order), and beat name. The *targetBeat* will be branched to as soon as the animation has finished playing.



EndScreen

The *EndScreen* beat function displays a title (something like “The End”) and a button to replay the story. Clicking this button sends the interactor to the *targetBeat* and resets all variables and counters to their initial values.

Edit beat 202 Beat 202

Graphics Editor

id: 202

name: Beat 202

Default targetBeat: undefined

Delay before default targetBeat: 0

Beat Function: endScreen

title The End

button Another Go

targetBeat: 1: Beat 1

Cancel Save Changes

The *Edit Beat* window for the *EndScreen* beat lets you set the beat ID number (used to determine default beat order), beat name, *default targetBeat*, *title*, a label for the button, and a *targetBeat*



RandomTarget

The *randomTarget* beat is an invisible beat function. It allows you to choose a set of possible *targetBeats* to send the reader to, out of which one will be randomly chosen.

id:

name:

Beat belongs to cluster:

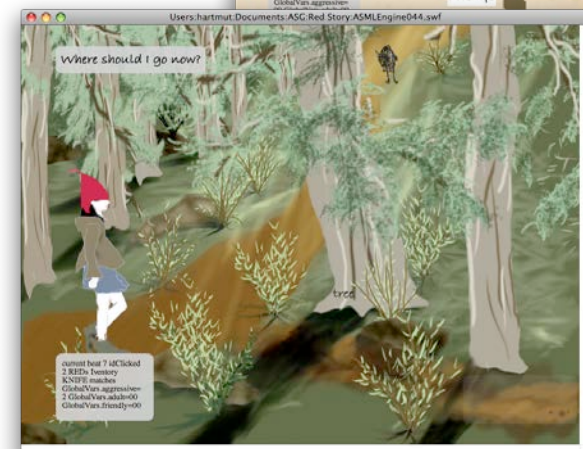
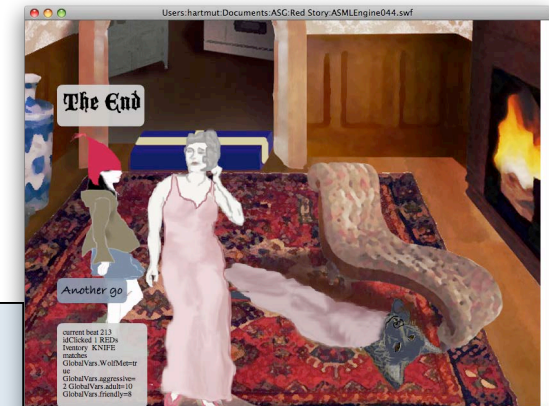
Beat Function: randomTarget

Choice ID 1:
targetBeat:

Choice ID 2:
targetBeat:

Choice ID 3:
targetBeat:

The *Edit Beat* window for the *randomTarget* beat lets you set the beat ID number (used to determine default beat order), beat name, the cluster to which the beat belongs, and the *targetBeats* that the reader could be sent to. Click the Add Choice button to add another option.



SetGlobal

SetGlobal is an invisible beat function. It is used to set a variable to a specific value. The typical usage for this beat is to record actions taken by the interactor, eg when the interactor has found a certain prop or met a character. A ConditionsCheck function can then be used later to check if the variable has been set to specific value. Setting of a variable with this function is completely invisible to the interactor.

The screenshot shows a window titled "Edit beat 10 Beat 10". Inside the window, there are several input fields and a dropdown menu. The fields are labeled as follows: "id:" with a text box containing "10"; "name:" with a text box containing "Beat 10"; "Beat Function:" with a text box containing "setGlobal"; "name:" with a text box containing "WolfMet"; "val:" with a text box containing "true"; and "targetBeat:" with a dropdown menu showing "11: Beat 11". At the bottom of the window, there are two buttons: "Cancel" and "Save Changes".

The *Edit Beat* window for the *SetGlobal* beat lets you set the beat ID number (used to determine default beat order), beat name, the *name* of the variable to be set, and the *value* that is assigned to the variable. You also need to set a *targetBeat*.

If the variable does not exist yet, it will be created for you.

setCounter Beat

The *setCounter* beat is another invisible beat. It automatically sets one counter to a specific value without any user interference. If you need to set multiple counters, you can create several of these beats and link them together.



id: 136

name: soldierGunReputation

Beat belongs to cluster: GiveSoldierV ▼

Beat Function: setCounter

Counter to set or change: SoldierFrienc ▼

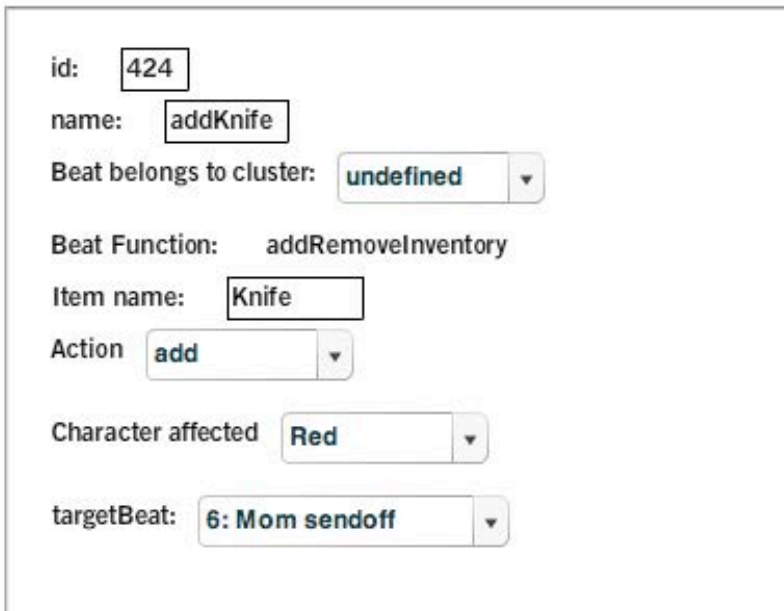
Counter Value: 4

targetBeat: 110: giveSoldierGun ▼

The Edit Item window for the *setCounter* beat lets you set the beat ID number (used to determine order), the beat name, the cluster in which the beat belongs, the counter to set, and the *value* you wish to set the counter to. You will also need a *targetBeat*.

AddRemoveInventory

The AddRemoveBeat is another invisible beat. It allows you to create items that can be given to or taken from any character in the story.



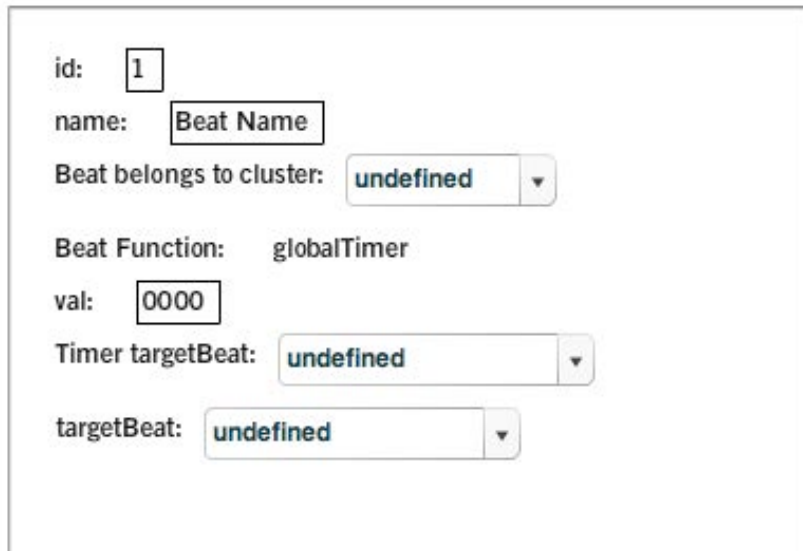
The screenshot shows a form for editing an item. The fields are as follows:

- id: 424
- name: addKnife
- Beat belongs to cluster: undefined (dropdown menu)
- Beat Function: addRemoveInventory
- Item name: Knife
- Action: add (dropdown menu)
- Character affected: Red (dropdown menu)
- targetBeat: 6: Mom sendoff (dropdown menu)

The Edit Item window for the *addRemoveInventory* beat lets you set the beat ID (used to determine order), the beat name, the cluster in which the beat belongs, the *name* of the item to be transferred, the *action* that will be conducted upon the item (i.e whether you are adding or removing it from someone's inventory), and the *character* that will be affected by this action. You will also need a *targetBeat*.

globalTimer

The *globalTimer* function is an invisible beat. It allows you to start a timer that counts down to zero as the player progresses with the story.



The screenshot shows the 'Edit Item' window for a beat named 'globalTimer'. The fields are as follows:

- id:** 1
- name:** Beat Name
- Beat belongs to cluster:** undefined (dropdown menu)
- Beat Function:** globalTimer
- val:** 0000
- Timer targetBeat:** undefined (dropdown menu)
- targetBeat:** undefined (dropdown menu)

The Edit Item window for the *globalTimer* beat lets you set the beat ID (used to determine order), the beat name, the cluster in which the beat belongs, and the length of the timer (measured in milliseconds – 1000=1 second).

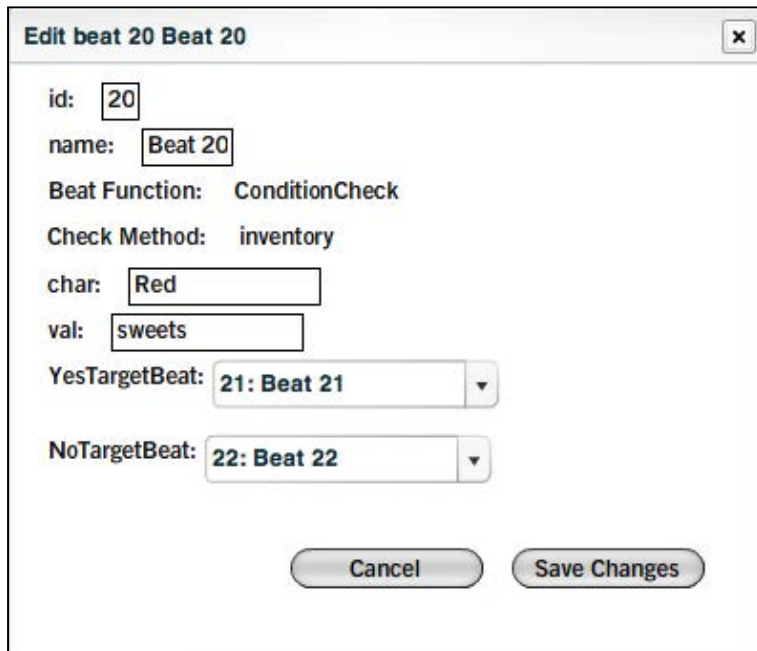
You need two *targetBeats* for this function. The *timer targetBeat* is the beat that the story will move to when the timer runs out. The *targetBeat*, however, indicates the next beat in the sequence.

ConditionCheck

ConditionCheck is another invisible beat function. This is a very flexible beat function, which is used to check a variety of values, and take different branching actions accordingly. *ConditionCheck* can check for inventory items, the id of the last Choice the interactor made, the value of variables set with *SetGlobal*, counter values, and timer values. Several *ConditionCheck* beats can follow each other, to create complex checks.

Inventory

This *ConditionCheck* beat checks for the presence of items in a character's inventory.



The screenshot shows a window titled "Edit beat 20 Beat 20". Inside, there are several input fields and dropdown menus. The "id" field contains "20". The "name" field contains "Beat 20". The "Beat Function" is set to "ConditionCheck". The "Check Method" is set to "inventory". The "char" field contains "Red". The "val" field contains "sweets". The "YesTargetBeat" dropdown menu shows "21: Beat 21". The "NoTargetBeat" dropdown menu shows "22: Beat 22". At the bottom, there are two buttons: "Cancel" and "Save Changes".

The *Edit Beat* window for the *inventory ConditionCheck* beat lets you set the beat ID number (used to determine default beat order), beat name, the name of the *character* whose inventory will be checked. The name of the prop you want to check for must be entered in the *val* field. Select the YesTargetBeat as the destination to branch to if the item is present, and the NoTargetBeat as the destination, if the item is not found.

Global

This *ConditionCheck* beat checks for the value of variable created with *SetGlobal*.

The screenshot shows a window titled "Edit beat 32 Beat 32". Inside, there are several input fields and dropdown menus. The "id" field contains "32". The "name:" field contains "Beat 32". The "Beat Function:" is set to "ConditionCheck". The "Check Method:" is set to "global". Below that, there is a "name:" field containing "WolfMet" and a "val:" field containing "true". There are two dropdown menus: "YesTargetBeat:" set to "33: Beat 33" and "NoTargetBeat:" set to "34: Beat 34". At the bottom, there are two buttons: "Cancel" and "Save Changes".

The *Edit Beat* window for the *global ConditionCheck* beat lets you set the beat ID number (used to determine default beat order), beat name, the *name* of the variable to be checked. The value to be checked must be entered in the *val* field. Select the YesTargetBeat as the destination to branch to if the value exists, and the NoTargetBeat as the destination, if the value cannot not found.

idClicked

This *ConditionCheck* beat checks for the ID of the choice made by the interactor in the preceding beat. Every choice in *ConversationChoice*, *MovementChoice*, and *PickProp* beats has an associated id set in the EditBeat window. This *ConditionCheck* can be used to branch to different targets based on this id.

Edit beat 8 Beat 8

id:

name:

Beat Function: ConditionCheck

Check Method: idClicked

val:

targetBeat:

val:

targetBeat:

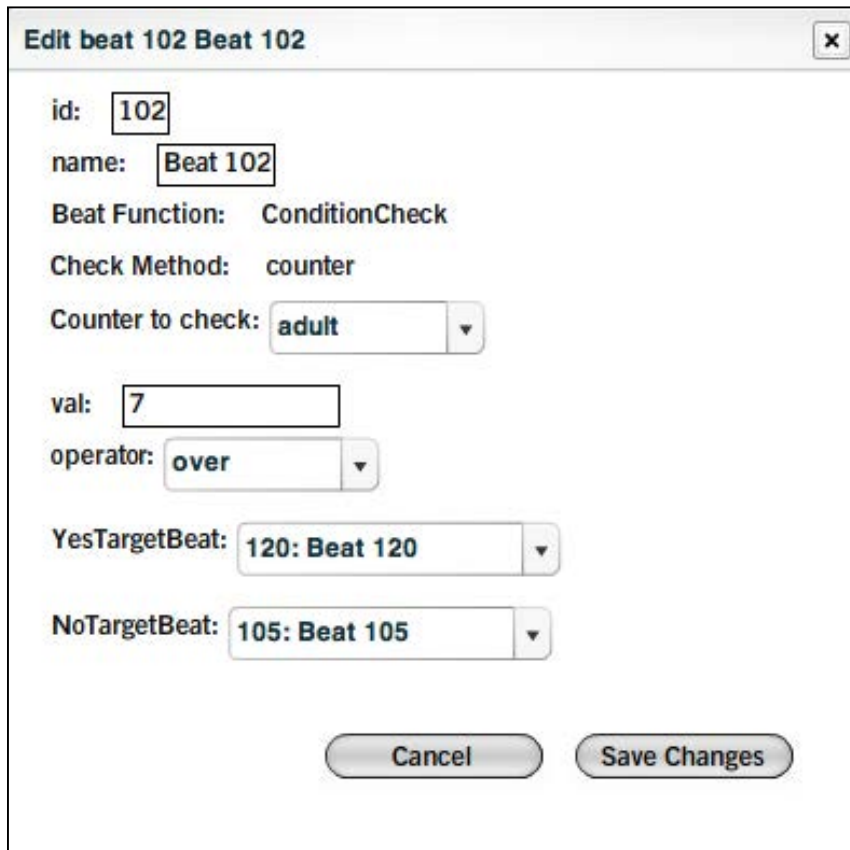
val:

targetBeat:

The *Edit Beat* window for the *idClicked* *ConditionCheck* beat lets you set the beat ID number (used to determine default beat order), and the beat name. The *val* fields correspond to the id of the choices in the preceding beat. Select the targetBeat as the destination to branch to for each choice.

Counter

This *ConditionCheck* beat checks for the value of a counter used for tracking a character's development. The current value is evaluated with an operator against a pre-set number, eg is the counter xy higher (over) than 14.



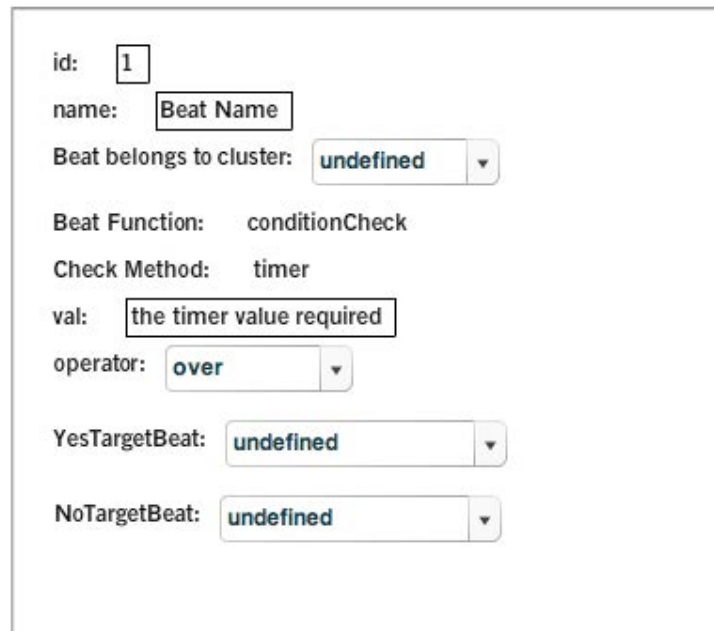
The screenshot shows a window titled "Edit beat 102 Beat 102". It contains the following fields and controls:

- id:** 102
- name:** Beat 102
- Beat Function:** ConditionCheck
- Check Method:** counter
- Counter to check:** adult (dropdown menu)
- val:** 7
- operator:** over (dropdown menu)
- YesTargetBeat:** 120: Beat 120 (dropdown menu)
- NoTargetBeat:** 105: Beat 105 (dropdown menu)
- Buttons:** Cancel and Save Changes

The *Edit Beat* window for the *counter ConditionCheck* beat lets you set the beat ID number (used to determine default beat order), and the beat name. Select the counter to change from the drop-down-menu. The *val* field holds the numeric value to compare against. You also need to select an operator, possible values are *over*, *equal*, and *under*. Select the YesTargetBeat as the destination to branch to if the equation is true, and the NoTargetBeat as the destination, if the equation is false.

Timer

This function of *conditionCheck* checks the current position of the timer currently running.



The screenshot shows the 'Edit Beat' window for a 'time ConditionCheck' beat. The fields are as follows:

- id:** 1
- name:** Beat Name
- Beat belongs to cluster:** undefined (dropdown menu)
- Beat Function:** conditionCheck
- Check Method:** timer
- val:** the timer value required
- operator:** over (dropdown menu)
- YesTargetBeat:** undefined (dropdown menu)
- NoTargetBeat:** undefined (dropdown menu)

The *Edit Beat* window for the *time ConditionCheck* beat lets you set the beat ID number (used to determine default beat order), and the beat name.

The *val* field holds the numeric value to compare against. You also need to select an operator, possible values are *over*, *equal*, and *under*. Select the YesTargetBeat as the destination to branch to if the equation is true, and the NoTargetBeat as the destination, if the equation is false.

Sound in ASB

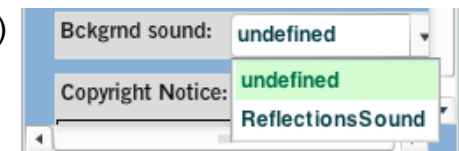
ASAPS supports sound in MP3 format. Soon, there will be a transition to OGG format, for licensing cost reasons.

You import mp3 files on the nodes/props panel, with the “New Sound” button.



Once imported, you can use a sound in 3 different places:

1. on the “settings” panel under *Bckgrnd sound* (this is the sound that plays all the time)



2. in a beat with clickable things (conversationChoice, pickProp, movementChoice, titleScreen, endScreen) as a button sound that plays if you click said thing



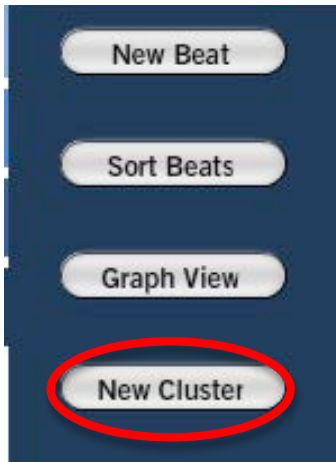
3. in an Edit Beat window, near the top, as sound that plays as long as the beat is on



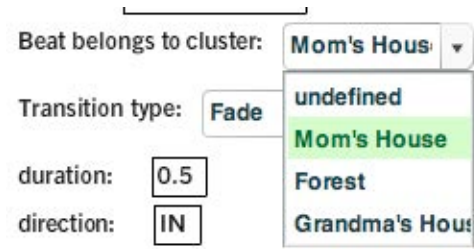
Clusters

If your story contains a large number of beats, you can organize them with the *Cluster* function.

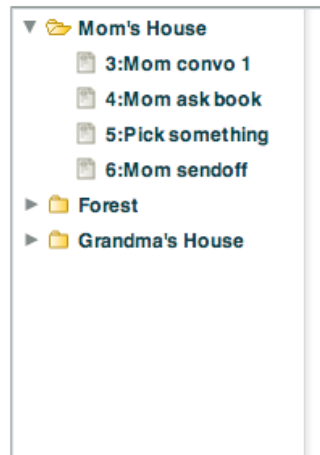
First, in the Narrative Design screen, click on the “New Cluster” button. Then, give the cluster a name.



Next, on the Edit Beat screen for any beat, assign that beat to the cluster you’ve created.

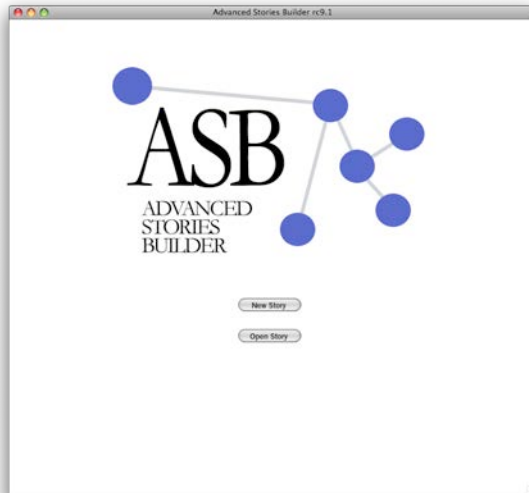


Now, on the sidebar of the Narrative Design screen, you can see which beats belong to each cluster.



Tutorials

Tutorial 1: Create a New Story



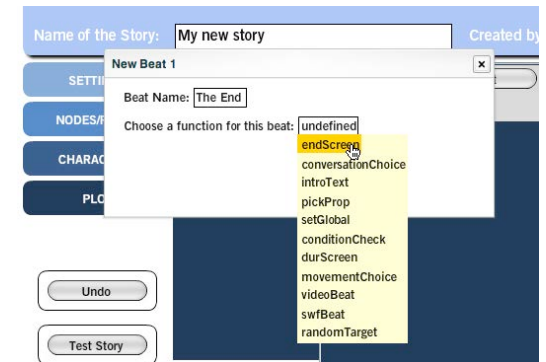
1. Open up the ASB software. On the splash screen, click the *New Story* button.

The dialogue box that pops up will ask you to pick a folder in which to save the new story. Be sure to pick one that does not have an existing ASML file, as it will be overwritten!

Once you've selected the destination folder, the Builder will save the ASML story file, as well as a copy of the current engine, at that location.



2. At the top of the screen, enter the name of your project in the text field at the left hand side, and the creator's name in the box at the right hand side.



3. The Builder starts you out with a TitleScreen beat. This is the first beat of your story.

Let's add an EndScreen beat, to finish the story. Click the *PLOT* tab at the left of the screen. Now click the *New Beat* button. Select *EndScreen* from the dropdown list. Enter "The End" in the Beat Name text field. Then click the *Save New Beat* button.

Edit beat 0 Start

Graphics Editor

id:

name:

Default targetBeat:

Delay before default targetBeat:

Beat Function:

Name of the Story:

Created by:

button:

targetBeat:

undefined

1: The End

0: Start

Cancel Save Changes

4. Now click the *Edit* button on the entry for the title Screen.

In the edit window select the endScreen beat with the name “The End” as the *targetBeat*. Then click the *Save Changes* button.

Edit beat 1 The End

Graphics Editor

id:

name:

Default targetBeat:

Delay before default targetBeat:

Beat Function:

title:

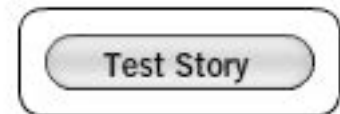
button:

targetBeat:

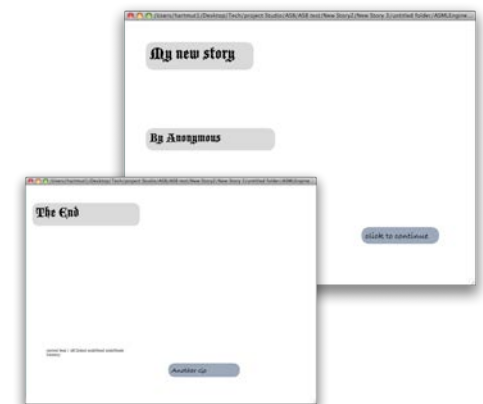
Cancel Save Changes

5. Click the *Edit* button for the endScreen beat.

In the edit window select the titleScreen beat with the name “Start” as the *targetBeat*. Then click the *Save Changes* button.

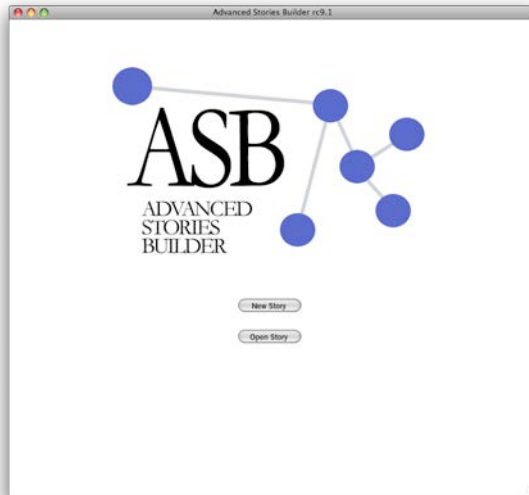


6. You can test your story from the Builder. Just click the *Test Story* button. The Builder will automatically save your story before launching it in the engine. Congratulations! You just finished your first project.

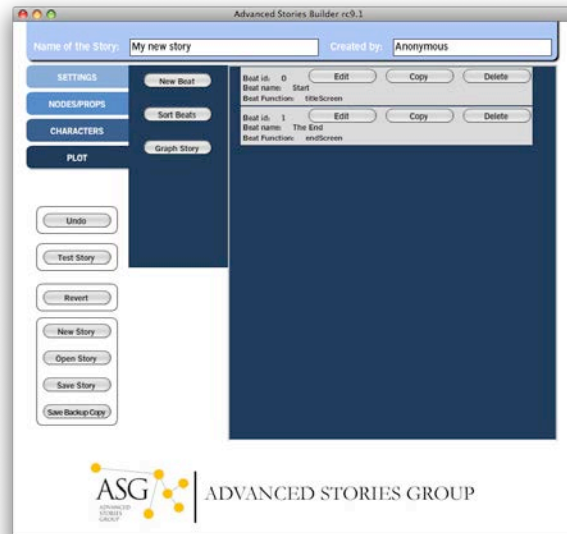


Tutorial 1: Your completed story will have 2 connected beats. The button on the title screen goes to the next beat by default. The button on the end screen restarts the story from the first beat.

Tutorial 2: Edit Colors and Text Styles in a Saved Story

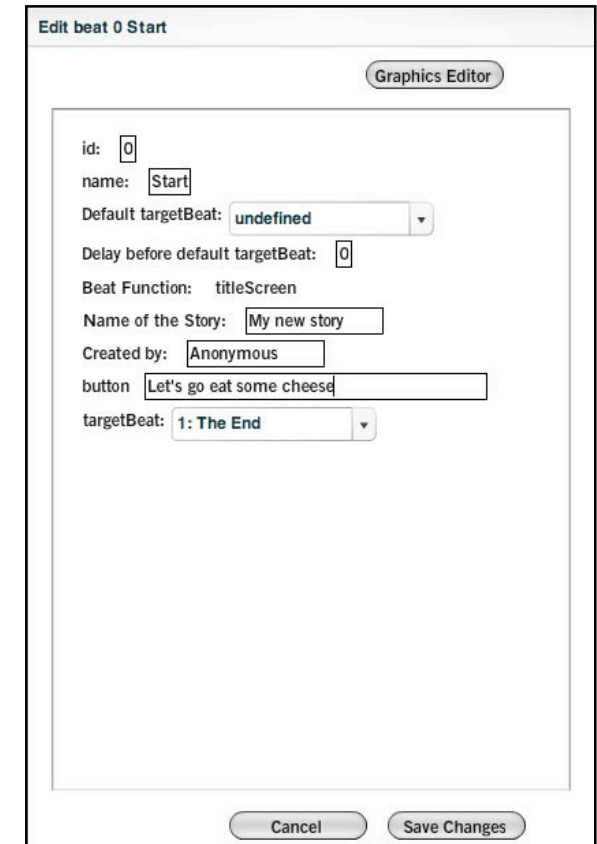


1. Open up the ASB software. On the splash screen, click the “Open Story” button. Select the file from Tutorial 1 in the dialogue box. It is called “Story.xml”. Depending on your Settings you might not see the “.xml” ending.



2. When you click on the *Narrative Design* tab this time, you’ll see the two beats you made previously.

Let’s edit the title screen, at beat 0, by clicking on the *Edit* button in its box.



3. The *Edit Beat* dialogue box lets you change the text that appears in the Title box and the continue button. The default title and creator text is the title you entered in the field at the top of the Builder screen previously. The default button text is “Click to Continue.”

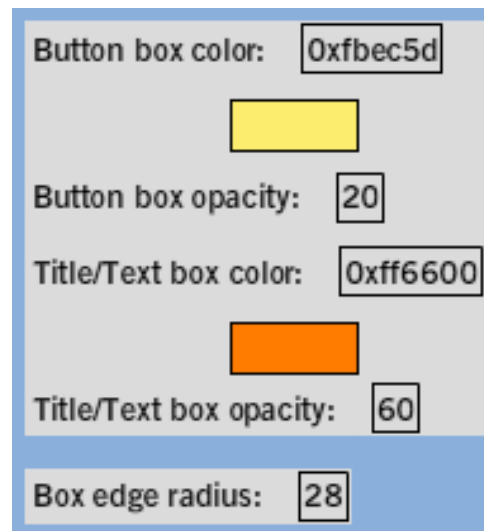
Let’s edit the button text. In that field, enter, “Let’s go eat some cheese!” then click the *Save Changes* button.



4. The Builder lets you customize the appearance of the text it generates as well as the content. You can specify the color, opacity and angle size of the boxes and buttons, as well as the font for the text inside.

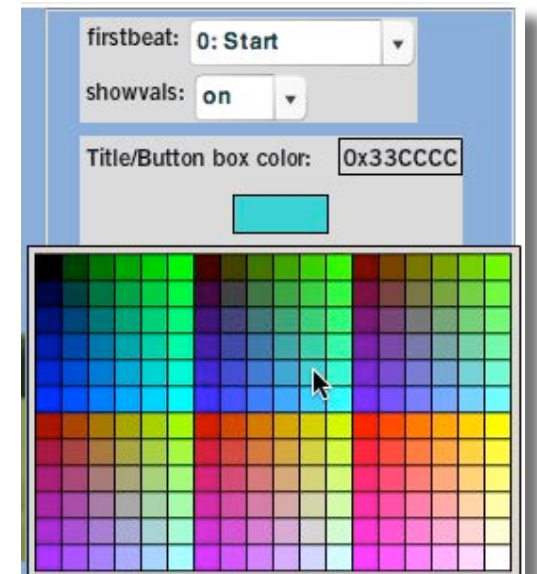
Let's give a new look to our story text. Click the *SETTINGS* tab at the side of the screen. You'll see the default settings displayed.

Let's make the boxes for our title and buttons a rich cheddar orange with a little translucence, and really rounded edges. The *Title/Text box color* and *Title/Text box opacity* fields determine the display for these 2 elements. Type in the new hex color value "ff6600" after the "0x" in the *Title/Text box color* field. Set the *Title/Text box opacity* value to "60". Set the *Box edge radius* value to "28".



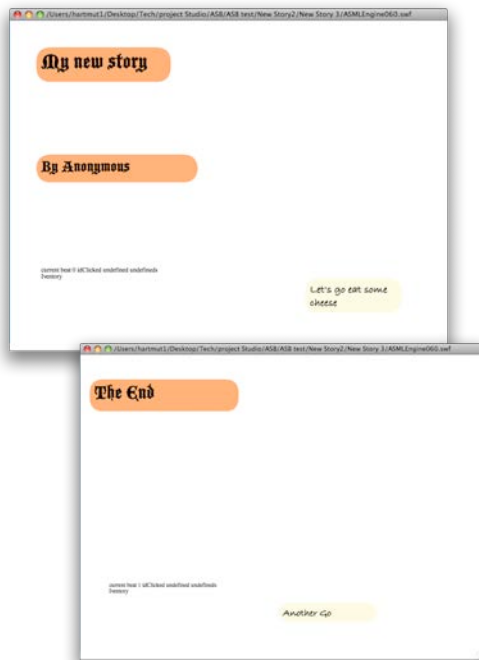
5. The box that contains the expository text can also be customized, through the *Button box color* and *Button box opacity* values.

Let's make ours a pale yellow with lots of transparency. Change the value after the "0x" in the *Button box color* field to "fbec5d", and set the *Title/button box opacity* value to "20".



6. Alternatively you can change the color settings by clicking on the rectangle below the hex value field. A color picker appears and you select a color by clicking.

While this method is easier, it only gives you access to a limited amount of colors. Inputting a hex value gives you access to the full color range. You can find Hex color values by using a software like Adobe Photoshop, or the online service Kuler (<http://kuler.adobe.com>)



7. Click the *Test Story* button. Your story now has a whole new look..



8. The Builder includes several different fonts that can be rendered by the engine. You can set a different font to the 3 kinds of text (title, text, and button) that are generated.

Let's mix things up a bit. Pick a different font for each of the text types from their respective dropdown menus. (The example above uses "Traditional" for the title, "Modern" for the expository text, and "Handwriting" for the buttons.)

Run your story by clicking on the *Test Story* button to see how your changes look in action.



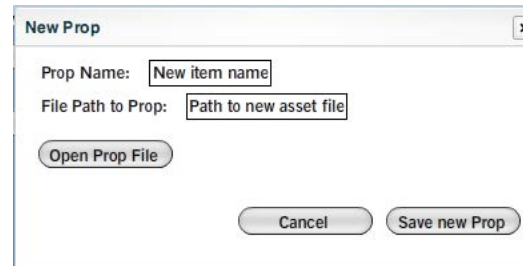
9. The result changes your story's look even more.

Changing color, fonts, and the shape of the boxes can dramatically alter the feel and tone of your work

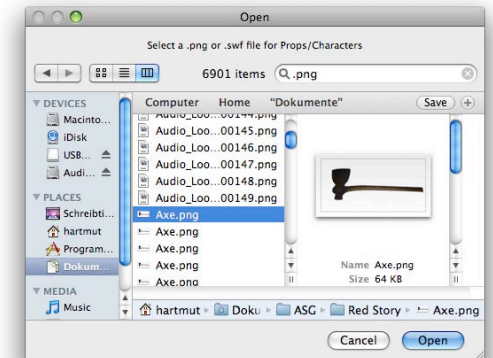
Tutorial 3: Adding Nodes and Props



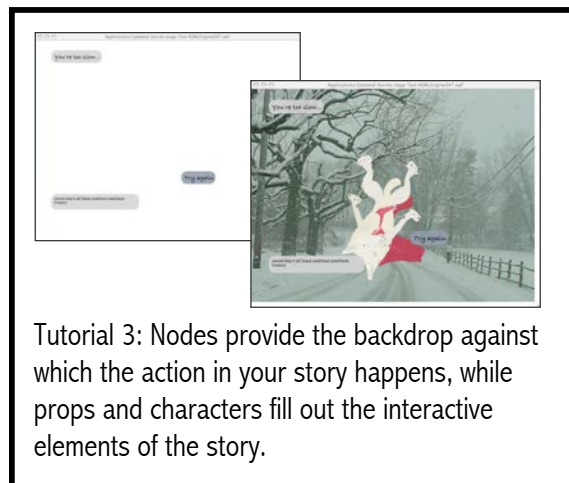
1. Click the *Nodes/Props* tap in ASB. Then click on the *New Prop* Button.



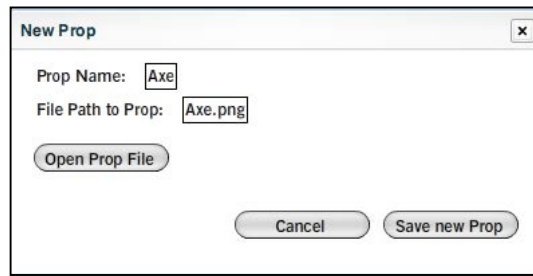
2. The *New Prop* dialogue appears. Click the *Open Prop File* Button here.



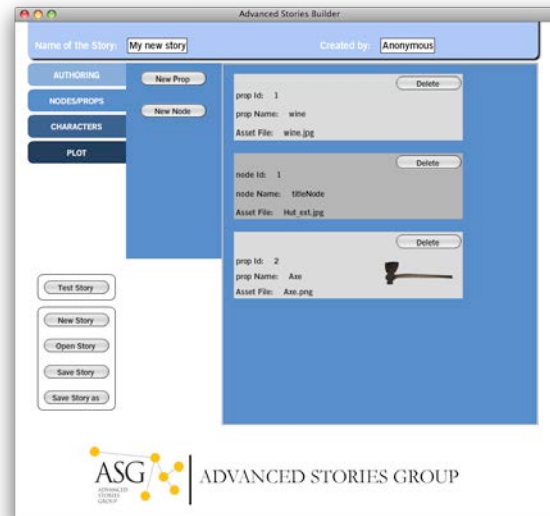
3. A file dialogue appears. Select either a PNG image or a SWF file. Click *Open*. ASB will copy the file for you to the current story folder automatically.



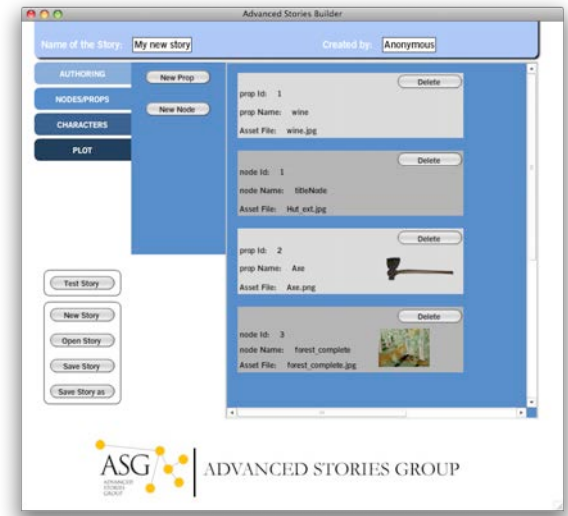
Tutorial 3: Nodes provide the backdrop against which the action in your story happens, while props and characters fill out the interactive elements of the story.



4. You can now change the name of the prop in the *New Prop* dialogue. **Do not change the file name.** Click the *Save new Prop* Button.



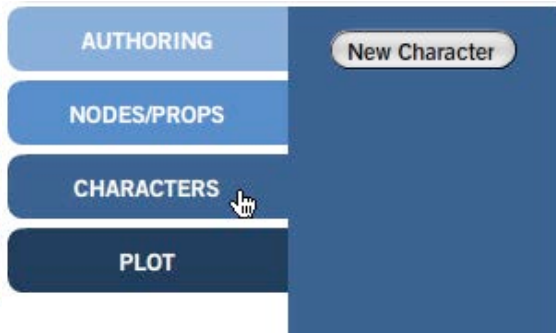
5. The new prop will appear in the list on the Nodes/Props screen.



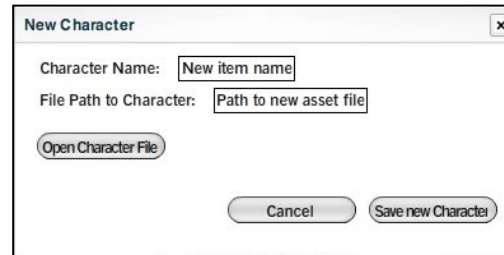
6. Adding a new node works pretty much the same way. Node images should fit the standard stage size of 800X600 pixels. Start by clicking the *New Node* button, then click *Open Prop File*. This time select either a JPG image, a SWF file, or a video file in the Flash Video format (.flv). Click *Open*. Change the name (and not the file path!) if you want. Click *Save new Node*. The new Node appears in the list.

Repeat this process for all your nodes and props. Then go to the next tutorial.

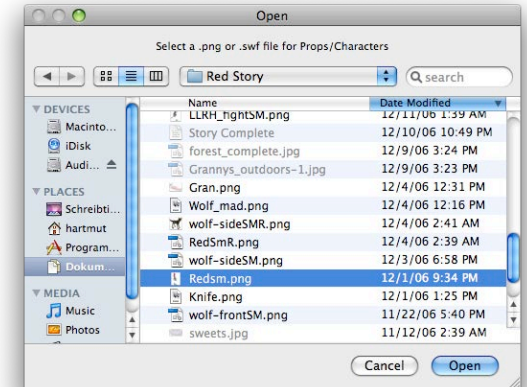
Tutorial 4: Adding Characters



1. Click the *Characters* tap in ASB. Then click on the *New Char* Button.



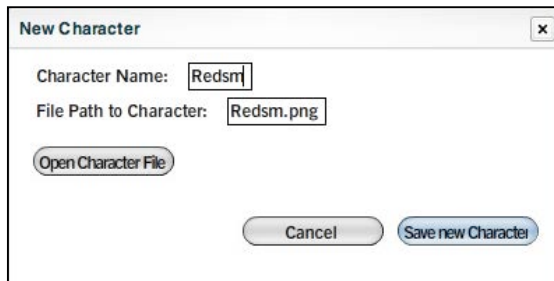
2. The *New Character* dialogue appears. Click the *Open Character File* Button here.



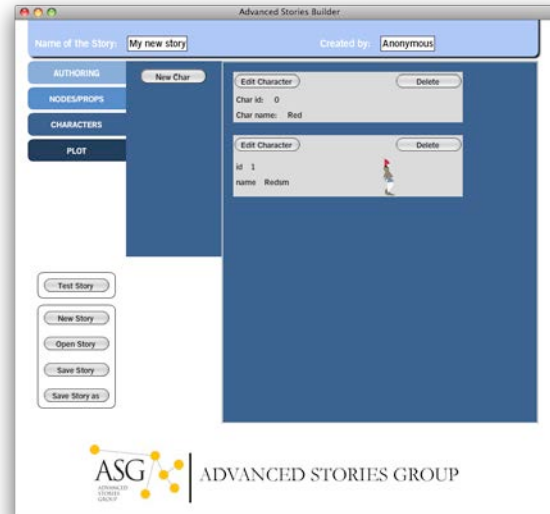
3. A file dialogue appears. Select either a PNG image or a SWF file. Click *Open*. ASB will copy the file for you to the current story folder automatically.



Tutorial 4: Adding characters makes your story more dramatic and expressive.



4. You can now change the name of the character in the *New Character* dialogue. **Do not change the file name.** Click the *Save new Character* Button.

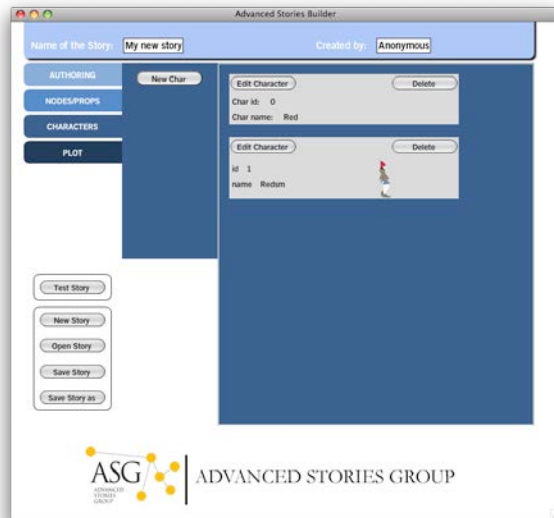


5. The new character will appear in the list on the characters screen.

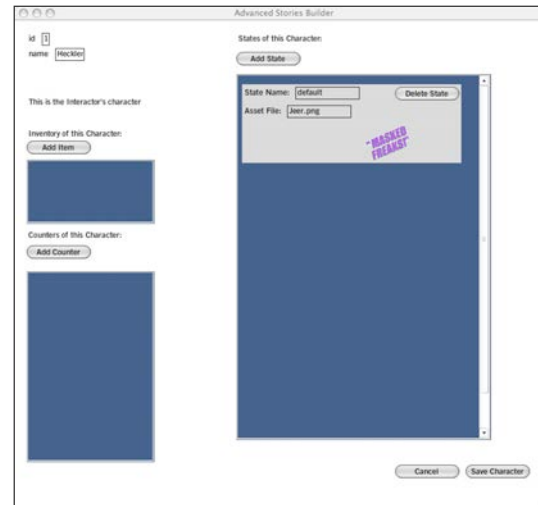


6. To add inventory items, counters, and states to your character, click the *Edit Character* Button. Refer to the *Character Editor* page in the *Features Overview* Section of this User Guide for a description of the *Character Editor*. The next tutorial shows you how to use these features.

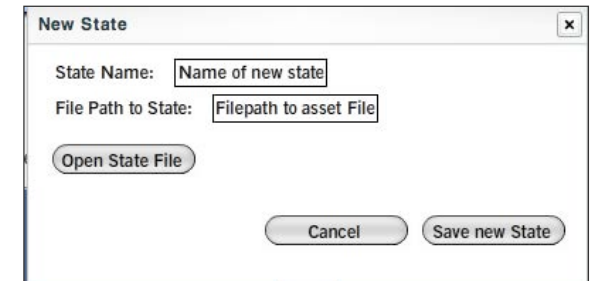
Tutorial 5: Editing Characters



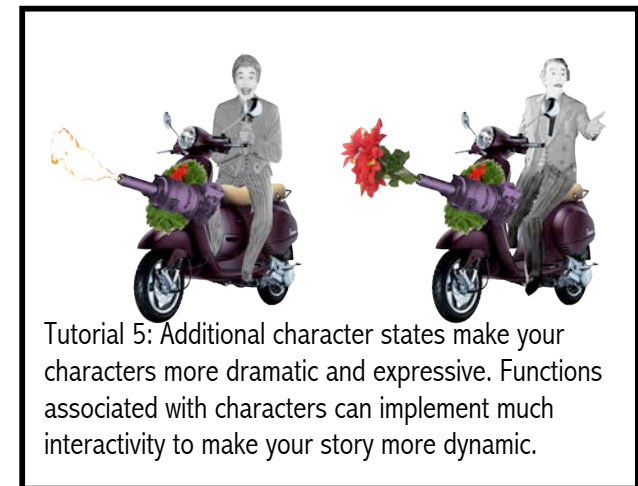
1. Click the *Character* tab in ASB. Then click on the *Edit Character* button in one of the character boxes to the right.



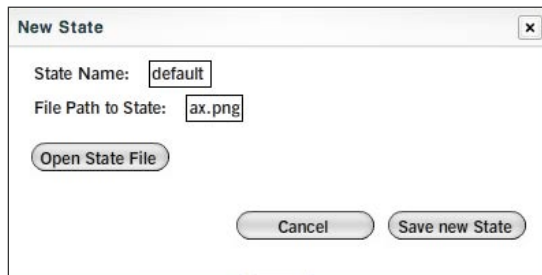
2. The *Character Editor* screen appears. Let's change the default image for this character. Click the *Delete State* button in the default character state box, and then click the *Add State* button at the bottom of the screen.



3. Click the *Add State* button at the bottom of the screen. The *New State* dialogue box pops up.

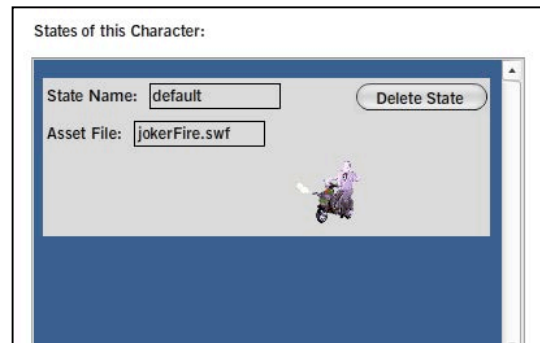


Tutorial 5: Additional character states make your characters more dramatic and expressive. Functions associated with characters can implement much interactivity to make your story more dynamic.



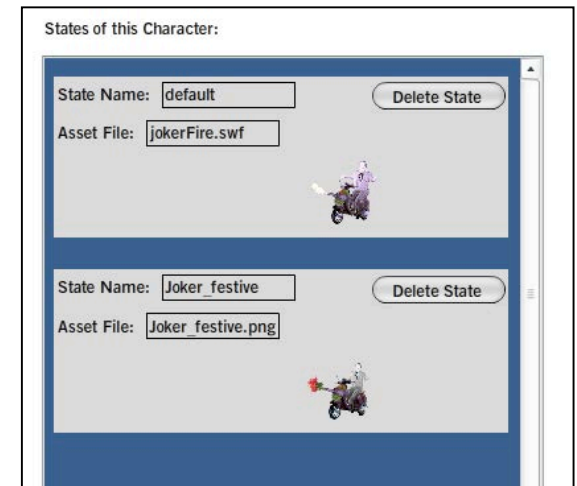
4. Click the *Open State File* button to select the image from your files. Since this will be the normal state for your character, change the name in the *State Name* box to *default*. Do not change the file path name.

Click *Save New State*.

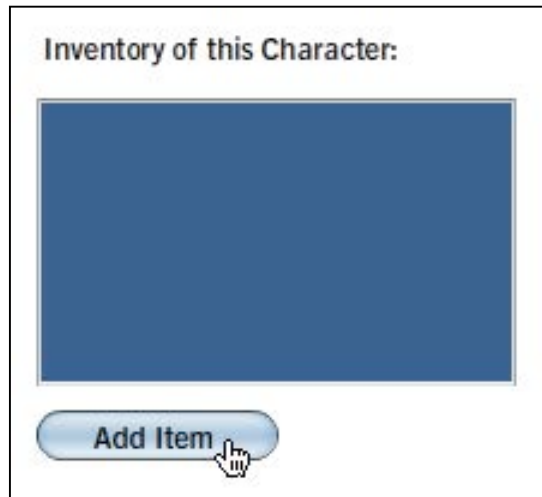


5. The new character state will appear in the character state list.

Let's add another state. Repeat steps 3 and 4, but this time name your character state anything except "default".



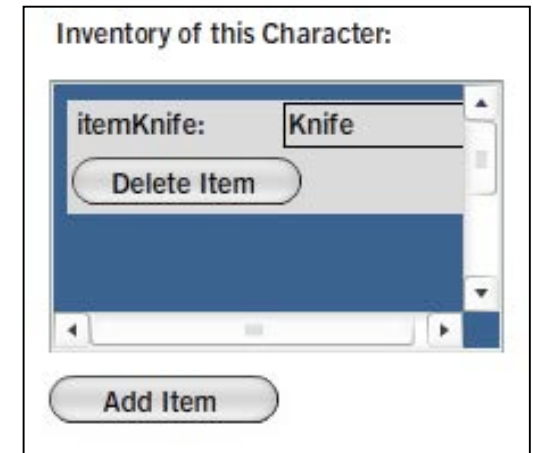
6. Additional states will appear in the list. You'll be able to select these states in the *Graphics Editor* (see next tutorial). For now, we will add inventory items and counters.



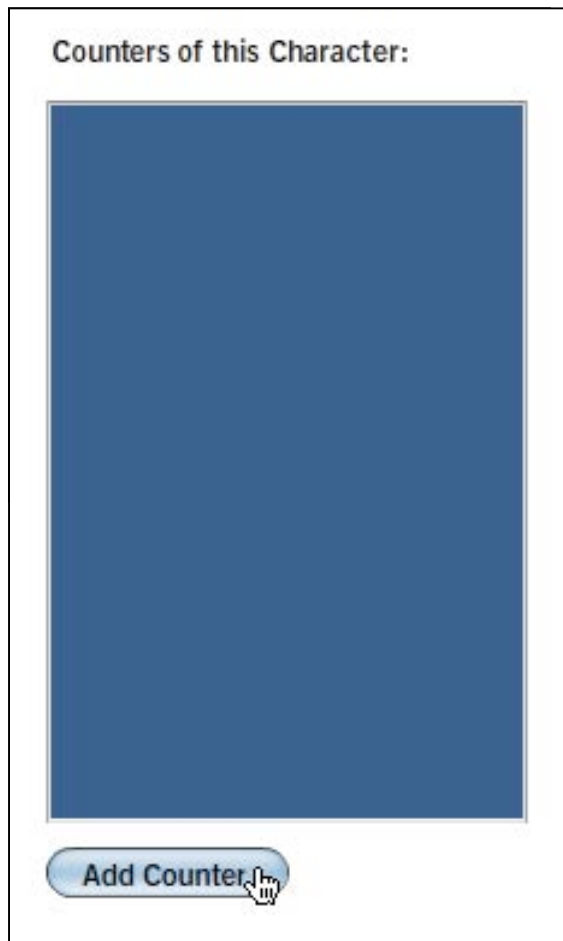
7. Click the *Add Item* button.



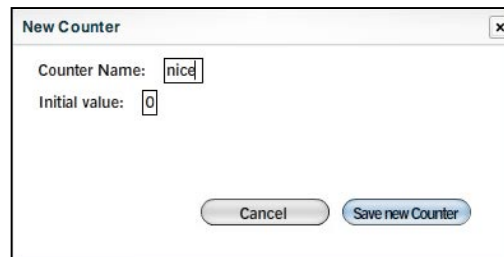
8. In the *New Item* Dialog, name your item, eg. *Knife*. Click the *Save new item* button.



9. The new item appears in the list. You can now add more items if you want to.



10. Click the *Add Counter* button



11. In the *New Counter* Dialog, name your counter, eg. *nice*. Click the *Save new Counter* button.

Counters are an important part of your story. You can understand them as trackers for the development of your characters or their progress towards a certain goal. For example your character could start out as “blank” and could become either nasty or nice depending on the interactor’s choices that update the counters. You would implement this by having two counters, named *nasty* and *nice*.



12. The new counter appears in the list. You can now add more counters if you want to.

Tutorial 6: Editing Visual Beat Content

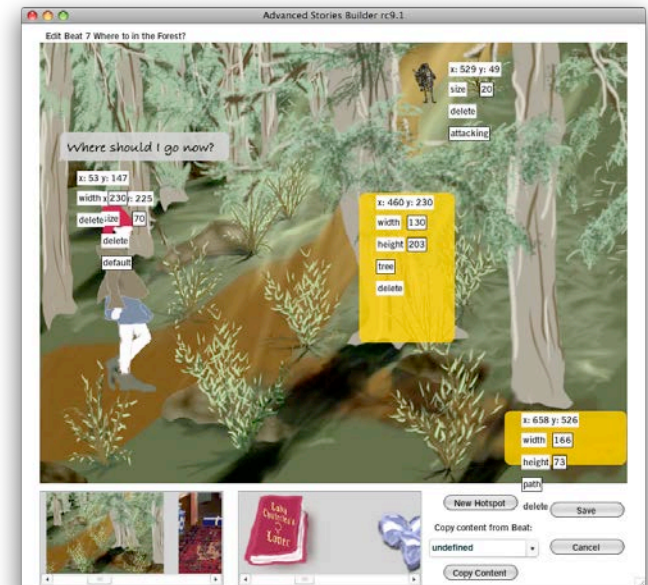


1. Click the *Plot* tap in ASB.



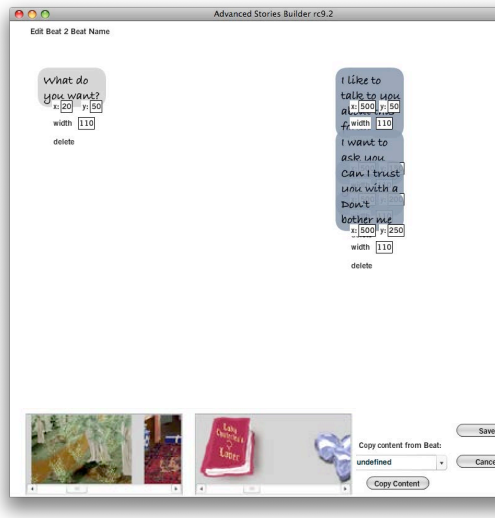
2. Click the *Edit* button for the beat you want to work on.

To use all aspects of the *Graphics Editor*, you need to add nodes, props, characters, and character states first – see tutorials 3,4, and 5. However, a story can work without any additional content. In this case, you can still use the *Graphics Editor* to place and adjust text boxes and hotspots.



3. Click the *Graphics Editor* button in the Beat Edit window to open the graphics editor.

The image above shows a beat already filled with content. Initially you will only see textboxes (see next step).



4. Text boxes appear automatically in the graphics editor.

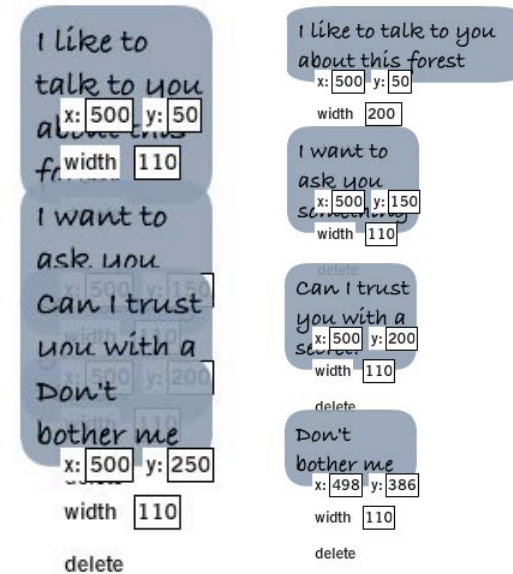
Depending on how much text you enter you need to change the *width* of a text box. Text will wrap automatically and thus adjust the height of a text box. If you want a single line of text, you need to make a text box wide enough to fit all text.

What do you want?

x: 20 y: 50

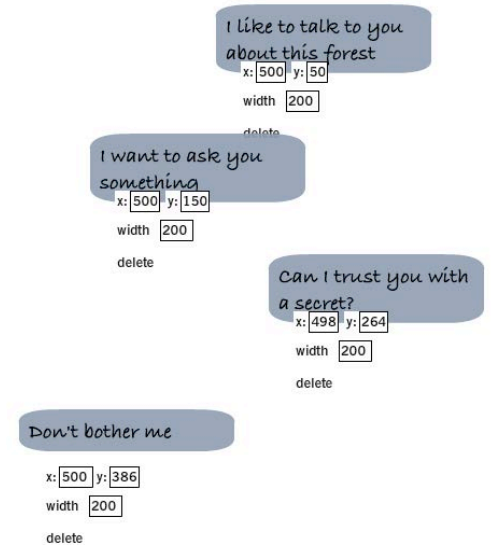
width 200

delete



5. In *conversationChoice* beats, if you have several choices and/or enter longer texts in the choices, the text boxes might overlap.

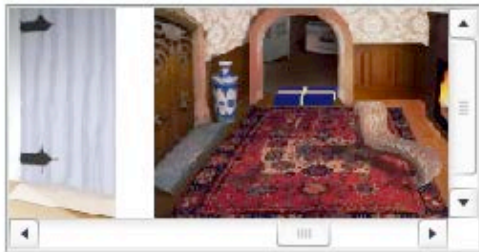
You need to adjust the *width* to fix this, as well as reposition the text boxes (see next step).



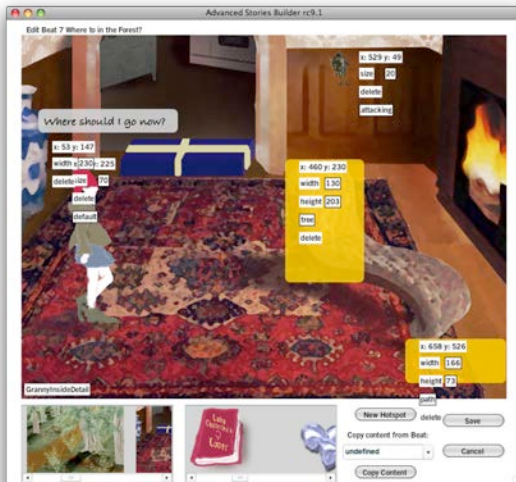
6. Click and drag to position a text box. You can also fill in values for x and y coordinates.

To remove a text box click the *delete* field.

Hint: Be aware that you can't bring back a deleted textbox other than using *undo* or *restore*.



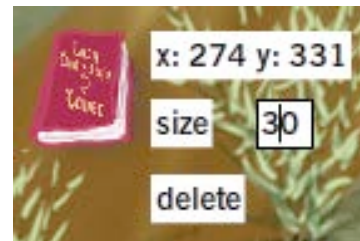
7. Use the the lower left scroller to change the background (node) of the beat. To change the node, click on one of the thumbnail images.



The beat from step 3 (previous page) with a different node.



8. Use the scroller in the lower middle of the Graphics Editor to add characters and props to the beat. To add a character or prop, click on one of the thumbnail images.



A prop on a node, resized to 30 percent.



9. Position a character or prop by clicking on it and dragging it around on the node. Click again to update x and y coordinates.

You can also fill in values for x and y coordinates in the respective text boxes.

Change the value in the *size* field to shrink or enlarge the character or prop.

If you want to remove a character or prop, click the *delete* field.



10. If a character has different states, you can change it by clicking on the state menu beneath the *delete* field and selecting a different state.

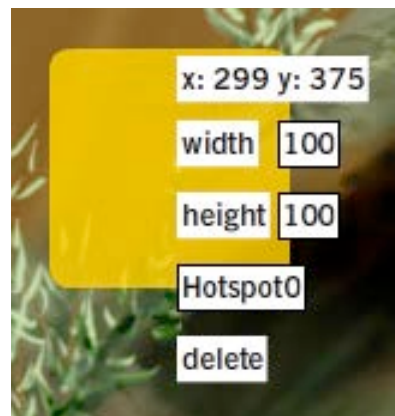
If you want to leave the menu without making changes, click again on the field.



11. In *MovementChoice* and *PickProp* beats, you can also add hotspots. These are clickable areas that overlay a node.

Click on the *Create Hotspot* button to create a hotspot.

Hint: The *Create Hotspot* button only appears in the two beat types mentioned above.

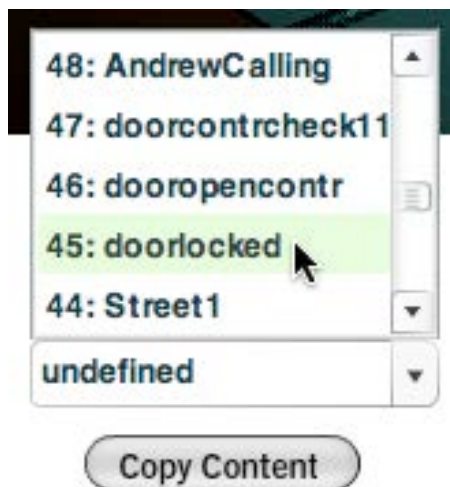


12. Click and drag to position the hotspot. You can also fill in values for x and y coordinates.

A new hotspot is named automatically (HotspotX). It is important to change this name in the text field, as the name is displayed when the user mouses over the field if *Hotspot Labels* is "on" in the settings. Hotspots are visible to the user if the *Hotspots Overlay* is set to "on".

Change the value in the *width* and *height* fields to adjust the size of the hotspot.

Click on *delete* to remove a hotspot.



13. If you want to create a beat with the same or similar visuals to an already existing beat, the most efficient way is to use the *Copy Content* function.

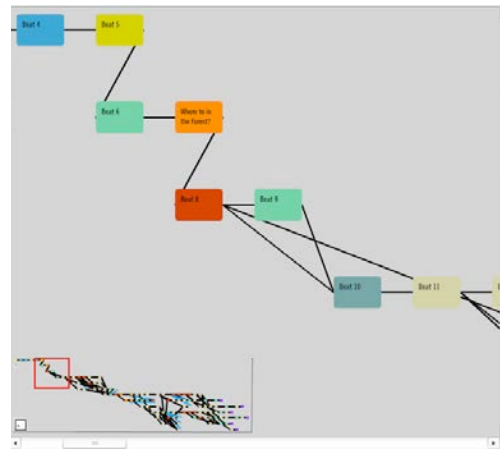
To use it, select a beat in the popup menu and click the *Copy Content* button.

Hint: Text boxes will not be copied between beats and hotspots will only copy between *movementChoice* and *conversationChoice* beats.

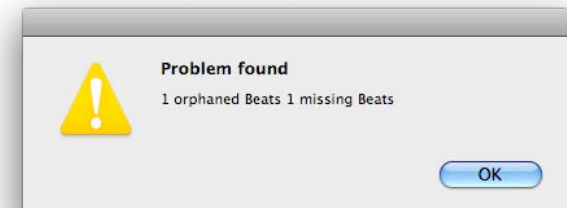
Tutorial 7: Working in Graph View



1. Click the *Graph Story* button in ASB.

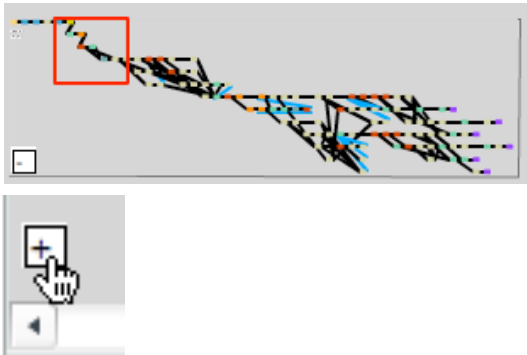


2. Calculating the Graph might take a little while, depending on the speed of your computer and the complexity of your story.

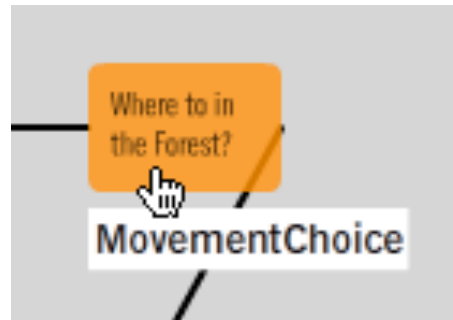


3. ASB warns you about orphaned (unconnected) and missing beats. Missing beats are referenced somewhere – for example as a targetBeat, but do not exist.

Orphaned and missing beats are listed on the left side of the story graph.



4. If the miniature view in the left lower corner gets in the way, you can hide it by clicking on the minus sign in the lower left corner. Clicking on the plus sign brings the miniature back.



5. Each beat displays the Beat Name and you see the beat function by moving (not clicking) your mouse pointer over a beat.

Hint: Different colored beats represent different beat functions, eg. orange = *movementChoice*.

Clicking on a beat brings up the edit window for the beat.



You can also create new beats in graph view. The new beat button moves to the left in this view.



6. Move your mouse pointer over a connection line between beats, and let it rest there for a very short while. A label comes up, listing and the start and end point of the connection. This is really handy if you have a lot of crisscrossing connections, which would otherwise be hard to identify.

The label disappears again if you hover the mouse pointer again for a little while.

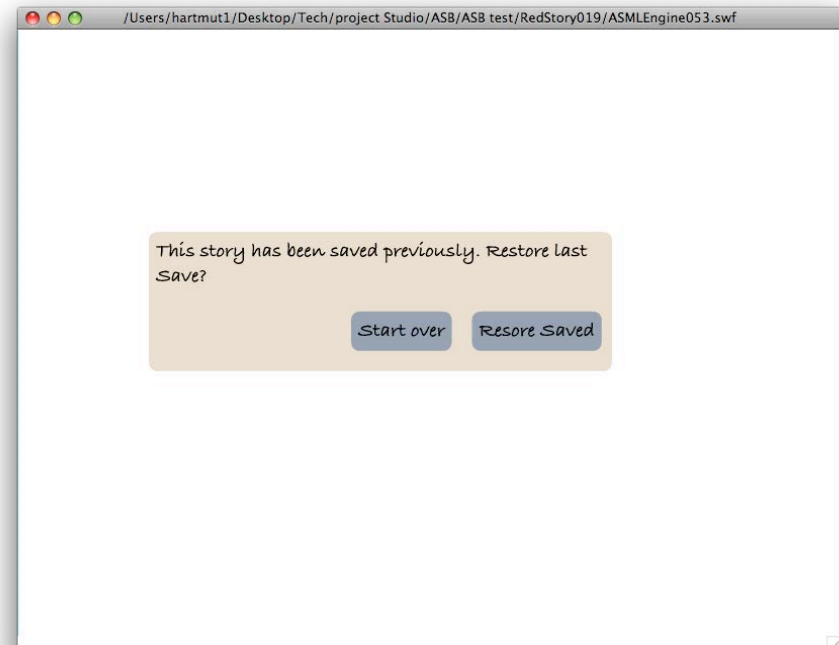
ASMLEngine Functions

The *ASMLEngine* presents your story to users. It does so in a way similar to a web browser, by reading the Story.xml file and putting the content together for any given beat.

Additionally, the *ASMLEngine* allows a user to save story and return later to the same beat in your story.



Pressing the "s" key on the keyboard saves the current state of a story. A dialog informs the user that the story has been saved



Reloading the same story now brings up this dialog, which offers a choice between *Start over* and *Restore Saved*.

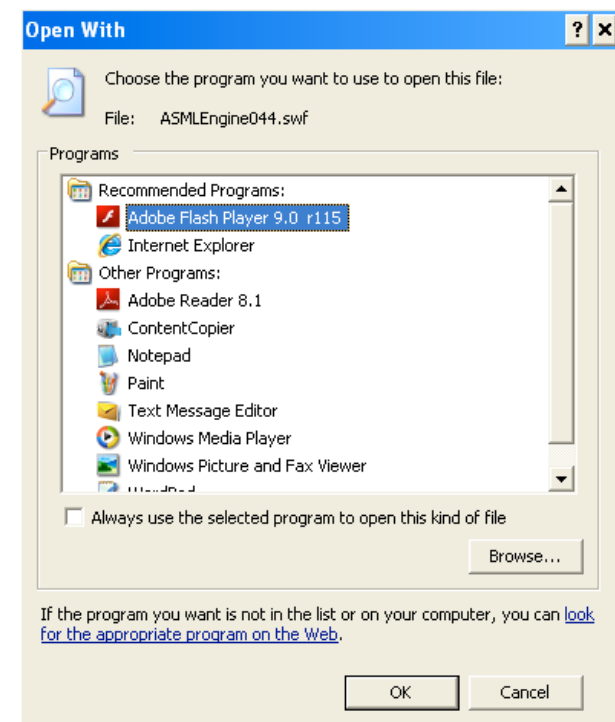
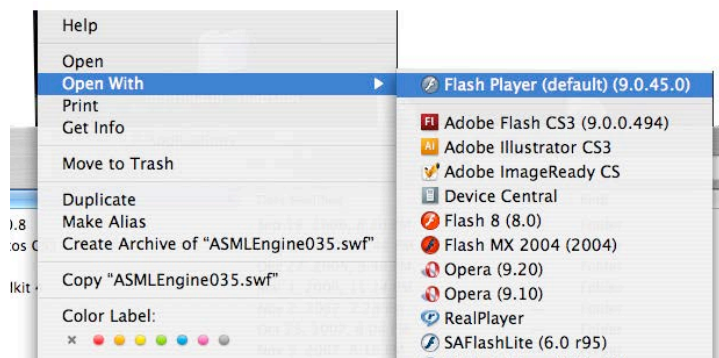
FAQs and Best Practices

Q: Why does nothing happen when I hit the “Test Story” button?

A. First, check to see that you have the Stand-alone Flash Player installed. This is different from the Flash browser plug-in. You can download the free player at http://www.adobe.com/shockwave/download/download.cgi?P1_Prod_Version=ShockwaveFlash. If you've already installed the player and nothing happens when you try to test your story, then you may not have set it as the default software for opening AdvancedStories .swf files.

Setting the Stand-alone Player to open AdvancedStories Engine files
Default (shown in OSX below and windows to the right):

2. Control-click (Mac) or Right-click (Win) the ASML Engine file
3. Click on “Open With” (Mac) or “Properties” (Win)
4. Select Flash Player from the list of suggested software. (Mac) or click on “Change...” (Win) and select the Flash Player from the list of suggested software.
5. (Win) if the player does not show up in the list, you need to click the “Browse” button and find the player on your hard drive



Q: Why does hitting the “Test Story” button cause the story to open in Flash as an editable file?

A. You need to set the Stand-alone Flash Player, not the Flash production environment, as the default application for .swf files. See above answer to solve this problem.

Q: Why did the builder put funny characters in my graphic file name when I uploaded it?

A. There was probably a space or another special character in the original file name. The builder escapes special characters (ie strips them out and replaces them with their ascii code characters) when it imports them into a story. There are technical reasons why the builder does this, but the ascii it inserts will not alter how your image file functions in your story.

Q: Why can't I rename the ASML story file to something other than “Story.xml”?

A: The ASAPS Engine uses this file name to recognize data files it can read. This is hard-coded in the software. The engine cannot read other file names, even if they are XML files that follow the ASML data schema.

Q: Why does each story need its own folder?

A: The Builder exports the ASML file with a specific file name that the engine uses, so there can only be one “Story.xml” per folder. Another important reason is the Builder creates self-contained stories that contain all the components to run a story, including the ASML data file, the assets, and latest engine build. This makes it really easy to share the stories you create with others.

Q: Can I change the beat function for a beat I've already made?

A: No, the Builder assigns the beat function when you initially create a beat. If you want to change how a beat functions, the best thing to do is make a new beat that utilizes that function. Don't forget to edit any beats that had led to or from that first beat. The Plot Story feature is enormously helpful for this sort of editing.

Q. Do I have to enter a story or creator name?

A: No. If you do not want the title screen to display a title and creator credit in dynamically generated text boxes, then delete the default contents in these text fields. However, if you leave one of these blank, they both must be blank. You cannot enter only a title or only a creator; in these cases the system will generate a text box on the title screen that reads “undefined.”

Q: Do I have to have a Title Screen as my first beat?

A: No, you can start your story with any beat type. You can either set the first beat by setting the Beat ID field of any beat (in the Beat Editor screen) to “0”, or you can use the firstbeat function (a pulldown menu in the Settings screen) to tell the engine which beat to start with.

Q: I changed the default character and uploaded a new image and renamed it. I see it in the Character Editor but not in the Graphics Editor. What is going on?

A: You can delete the original default character, but be aware that when you make a new character, the file name is used as the name unless you change it. In this case, you need to rename the normal character state to “default”. You can name other states for this character anything you like, but at least one must be “default”. Once you make this change and save, your new character should appear in the right-hand scroll box in the Graphics Editor.

Q: Can I use special characters in my file names?

A: You cannot use full stops (ie periods). You can use dashes and underscores, however.

Definitions

Beat: a beat is the unit of action in the AdvancedStories Narrative Model. Stories built in the ASB are broken out into a series of interconnected beats that vary by function.

Node: a node is the graphical backdrop for a beat. Nodes can be .flv, .swf, or .jpg files.

Hotspot: hotspots enable interactivity in MovementChoice. These are invisible and can be sized according to need.

Prop: a prop is any visual element that you place in a beat.

Character: characters can affect the path of the story. They can have different graphics associated with different states, as well as inventories of items and counters.

Target Beat: target beats continue the story path from a preceeding beat. A given beat may lead to multiple target beats, depending on its function. Beats that have not been assigned as targets will appear in the Plot Story screen as orphans.

Counter: counters are tied to characters and track specific values relating to user choice. They are a to model decision-making, consequences, relationships and causality with the ASB. You can create an unlimited number of counters. Counter values can be changed in individual beats.

Conditions: conditions include the different variables that change during a story. The ConditionCheck beat function evaluates the current story state for specific conditon status(es) and determines the target beat based on that information.

IdClicked: this condition checks to see if a specific prop or location has been chosen during the story play.

Counter: this condition checks for a specific counter value at that beat.

Inventory: the inventory stores items associated with a character. It can be pre-filled in the Character Editor, but a character can also add items to the inventory during story play using the PickProp function.

Global: globals have a similar function as counters, in that they can set thresholds and specific conditions for plot movement. Globals, however, are Booleans, and are not associated with a specific character.

Functions: functions are the mechanics of the story system. They determine what action can happen in a beat, and how that beat will be rendered. Some functions create “invisible” beats that are purely computational and have no visual assets associated with them. Invisible beats cannot be edited through the Graphics Editor.

IntroText: visual beat. This function creates a beat with a static text field at the top of the screen, and an interactive button at the lower right-hand side. The button moves the story to the target beat. Both field text and button text can be entered through the beat window.

MovementChoice: visual beat. This function provides spatial choice. You can place props or hotspots on the screen; each choice is assigned a target beat.

PickProp: visual beat. This function allows the player to select a prop on the screen. The prop then goes into the player’s inventory.

EndScreen: visual beat. This beat function lets your players know they’ve finished your story by including “The End” and provides a button to start a new session (with all variables and conditions reset to their initial values). You can customize the visual appearance of this beat in the Graphics Editor.

ConversationChoice: visual beat. This beat function simulates a conversation and creates provides interactivity through player response. The question text will appear on the same side of the screen as the designated Questioner. You can include up to 3 responses from which the player may choose. Each response can have a counter and value associated with it. Each response can also lead to a different target beat.

SetGlobal: invisible beat. This beat function establishes a global variable and value that can be checked throughout the story by means of the ConditionCheck beat function.

ConditionCheck: invisible beat. This function checks whether a specific condition has been met, and determines the next beat depending on the values you've provided. The possible conditions include idClicked, counter, global, and inventory.

idClicked: This checks to see if a specific id (from a hotspot/location or prop) has been selected at any point during play. Up to 3 values (and 3 respective target beats) can be set.

Counter: This checks for a specific counter and value. The threshold can be set for over, equal, or under that value. There are 2 possible target beats, reflecting whether that counter state has been satisfied or not.

Global: This checks for the state of a global variable. It is a true/false Boolean value that determines the next beat from 2 possibilities you set.

Inventory: This checks for the presence of a specific prop in any character's inventory. It is a true/false Boolean value that determines the next beat from 2 possibilities you set.

DurScreen: visual beat. This will display the beat for the chosen period of time before automatically proceeding to the target beat.

VideoBeat: visual beat. This function allows you to play a .vlf file automatically, without any input from the interactor. You upload the video file as a node. You can place props or characters on the screen; they will appear as static elements over the video background while the video plays. The story will automatically proceed to the target beat once the video clip has finished playing.

SWFBeat: visual beat. This function allows you to play a .swf file automatically, without any input from the interactor. You upload the .swf file as a node. You can place props or characters on the screen; they will appear as static

elements over the background while the file plays. The story will automatically proceed to the target beat once the movie clip has finished playing.

Interactor: the player character. Any character in a story can be set as the interactor, but a story can only have 1 interactor. Choosing the interactor affects some of the engine functions, such as where conversation choices appear in a screen.

Questioner: which character asks the question in the ConversationChoice beat function. This determines where the question text and the response text appear in the screen; the question text will appear on the same side of the screen as the character designated the questioner.

State: the different graphic files representing a character. A character can appear in different states in addition to the default state set when created. States are one way to make your characters more expressive and lifelike. You can use different states to reflect emotions, gestures, or movement, for example. You can also have character states that reflect possible items they may pick up in their inventories.

Inventory: an array of props associated with a character. You can put pre-place props in any character's inventory so that they will be there at the start of a story. You can also add props throughout the story to the interactor's inventory, using the PickProp function. The ConditionCheck beat function can trigger narrative choices and story events based on the presence (or absence) of a particular prop.

Story.xml: the file name for the ASML document that contains the story data. The ASB automatically gives this file name to any story it creates, so each story must reside in its own folder.

Contact

On the Web

<http://advancedstories.net>

Email

Info@advancedstories.net